Structured Scalar Potential : Design and Theory

compiled by D. Gueorguiev, 6/14/2026

Table of Contents

Notation and Terminology ***2***

Quantities shared with the SPLM deep dive **2**

Architecture 2

State variables 3

Parameters and derived quantities 3

Quantities specific to the structured potential **3**

Disambiguation: symbols that collide between the two documents **4**

Motivation ***5***

Potential Wells as Energy Landscapes **5**

Semantic Attractor Extraction Algorithm **5**

Potential Wells and the Concept of Temperature ***7***

Temperature as Boltzmann weight, softmax, and log-sum-exp **7**

What temperature does not change **7**

Three mixing regimes **8**

Unit temperature, responsibilities, and the force **9**

The mixing rule does not assume quadratic wells **10**

Quadratic and Gaussian Potential Wells ***10***

Quadratic Wells **11**

The equivalence 11

Why SQ3 recovers the Gaussian mixture 11

The parameter τ (temperature) in the quadratic well design 11

What τ controls 12

The practical punchline 12

Gaussian Wells **12**

Isotropic implementation: MixtureGaussianVTheta 13

Anisotropic implementation: diagonal + low-rank precision 14

Empirical comparison 15

Three weights that are easy to confuse 15

Why the distinction has teeth: truncation moves gk upward 16

Curvature Diagnostics and Rank Selection for the Anisotropic Well **17**

Effective rank as a measurement: the participation ratio 17

Rank under a binding cap: redistribution, not addition 18

A procedure for choosing the rank 18

Note on multi-ξ context channels and K-EMA **19**

Role of ξ in the Euler-Lagrange step 19

Single channel : rank-1 causal mean 19

K-EMA : K causal weighted averages at several horizons 20

Note on the hidden width of the MLP $V\theta$ **21**

How MLP $V\theta$ consumes the bank of context channels ξtk 21

Structured and Gaussian $V\theta$: flatten, then project **22**

Note on structured $V\theta$: Well Banks, Well Centres, Well Precisions, Well Mixture weights and their projections 22

Does the anisotropic Gaussian benefit beyond 4 Channels? 25

Can we do something better than flattening? 26

Depth conditioning and context banks in structured V theta **26**

Context banks 26

Depth codes as a per-layer shift 27

Where the forty wells sit before training 27

Analytic Multi-Channel Integration: a Ladder of Couplings **28**

The analyticity hinge 28
Product-of-experts fusion in closed form 28
The catch: fusion concentrates curvature 29
The harmonic split stays valid 29
The prototype: JointContextAnisotropicGaussianVTheta 30
What this predicts, and how to test it 30
References ***30***
Appendix ***30***
Note on logit ℓ and embedding matrix E in SPLM **30**
Note on SARF-faithful mass **32**
Component 1 — SARF-faithful ξ (§14.13) 33
Component 2 — logfreq per-token mass (§14.14) 33
Combined result (§4.4 of Training_and_Inference_with_SPLM.md) 33
Note on learning in Fock-PARFLM using structured $V\theta$ energy landscape **33**
Clarification on "optimised anchors" 34
What else carries gradients (everything in this list is trainable) 35
The tension 36
The conceptual point 39
Note on Why Tied LM Head worsens the Perplexity score **39**
The problem: tied head forces the dynamics to encode the frequency prior 40
D0.4 evidence 40
Why untying fixes it 40
1. Setup and notation 41
2. The column-space bottleneck 41
3. The optimal logit vector and the projection error 41
4. Decomposition: frequency prior + contextual signal 42
5. Gradient conflict for rare tokens 42
6. A lower bound on the excess CE from tying 43
7. Why this is worse for Fock-PARFLM than for transformers 43
8. Connection to D0.4 43

Notation and Terminology

This document shares its notation with the companion deep dive Deep_Dive_in_the_SPLM_model_and_code.docx. The entries in the first subsection below are reproduced from that document's "Notation and Introductory Notes" unchanged, so a symbol means the same thing in both places and neither document is the authority the other has to be checked against. The second subsection adds the quantities that exist only once a potential is *structured* into a bank of wells. The third records the handful of symbols that genuinely collide, which is the price of keeping each document's internal conventions intact.

Quantities shared with the SPLM deep dive

Architecture

d – embedding/hidden state dimension ; in code – `cfg.d`. Every tensor in SPLM lives in $\mathbb{R}^d$ or products of it thereof.
L – number of integration steps (shared-weight depth). Each step applies the same $V_\theta, \mathfrak{m}, \gamma$; in code `cfg.L`.
Δt – integration time step for the damped Stoermer-Verlet integrator; in code `cfg.dt`; Δt is fixed to 1.0.
K_ξ – number of multi-ξ context channels. When the context permits, we will use K instead of K_ξ to denote number of context channels.

K_w, K_{mix} – number of energy wells per bank; K_{mix} represents the older notation adopted when the scalar potential was constructed using mixture models (e.g. SQ3)
n_{ctx} – number well banks; usually $n_{\text{ctx}} = K_\xi$

State variables

$h_t^{(\ell)}$ - hidden state at integration step $\ell \in 0, \dots, L$. Sometimes we drop the subscript t - h_ℓ. Each h_ℓ is a position in semantic space ($\mathbb{R}^d$). In code – h. Shape: (B, T, d)
v_ℓ - velocity at integration step $\ell \in 0, \dots, L$. $v_\ell \coloneqq h_\ell - h_{\ell-1}$; in code – v. Shape: (B, T, d)
ξ_t – per position causal context vector. In the vanilla SPLM: $\xi_t = \frac{1}{t}\sum_{s=1}^{t} h_s$ – causal cumulative mean, computed once at layer $\ell = 0$. In the SARF variant $\xi_t^{(\ell)} = \frac{1}{t}\sum_{s=1}^{t} h_s^{(\ell)}$ (re-pooled from `h.detach()` at each step ℓ). In code: `xi` / `xi_now`. Shape: (B, T, d)
f_ℓ - conservative force at integration step ℓ. $f_\ell = -\nabla_h V_\theta(\xi, h_\ell)$. In code: `f`. Shape: (B, T, d)
$\nabla_h V_\theta$ – gradient of the scalar potential w.r.t. the hidden state, computed via torch.autograd.grad(V, h_in). The negation of this gradient is the conservative force f. In code: `grad_V`. Shape: (B, T, d)

Parameters and derived quantities

E – token embedding matrix; weights are tied: output logits are $h_L \cdot E^{\mathrm{T}}$. in code: `self.E`. Initialized with $\mathcal{N}(0, 0.02^2)$. Shape: `nn.Embedding(V, d)`
θ – all weights and biases of the scalar-potential MLP. These parametrize the energy landscape $V_\theta(\xi, h)$. In code: `self.V_theta.net.parameters()`
$V_\theta(\xi, h)$ - the scalar potential (energy) function. An MLP: Linear($2d \to d_V$) → GELU → [Linear($d_V \to d_V$) → GELU] × ($n_V - 1$) → Linear($d_V \to 1$); takes a concatenation $[\xi; h]$ and outputs a scalar energy. In code: `self.V_theta`. Shape: $\mathbb{R}^{2d} \to \mathbb{R}$
$\mathfrak{m}$ – semantic mass. Stored as a raw parameter; the physical mass is $\mathfrak{m} = \text{softplus}(\mathfrak{m}_{raw}) + \varepsilon$ to ensure positivity.
γ – damping coefficient, controls the rate at which velocity decays. Given with $\gamma = \text{softplus}(\gamma_{\text{raw}})$. Can be learned or fixed. In code : `self.gamma` / `self.raw_gamma` / `cfg.fixed_gamma`.
$V_\theta\left(\xi_t, h_t^{(\ell)}\right)$ - scalar potential energy evaluated at position $h_t^{(\ell)}$ with context ξ_t. In code: `self.V_theta(xi_now, h_in).sum()`. The `.sum()` aggregates over batch and position for a single backward pass; per-token gradients emerge because each h_t contributes independently to its own V_θ slice.

Quantities specific to the structured potential

A *structured* potential replaces the black-box MLP V_θ with an explicit sum over parameterised wells. The following quantities describe that structure; none of them exist in the vanilla SPLM, where V_θ is an MLP with no interpretable internal geometry.
k – well index, running over the K wells of a bank. A second index selects the bank when several are present.
τ – temperature of the mixture. Sets how sharply the wells compete: at $\tau \to 0$ the nearest well takes everything, at large τ the mixture flattens toward a single broad basin. In code: tau; the canonical value is $\tau = 1$, which matches energy units to probability units with no rescaling.
π_k, w_k – mixture weight of well k. π_k denotes the normalised prior in the log-sum-exp quadratic family; w_k the unnormalised amplitude in the Gaussian sum-of-bumps family. In code: w.
μ_k – centre of well k in hidden-state space, a point in $\mathbb{R}^d$ toward which the well attracts. Projected from the context ξ, never from h. In code: mu.
a_k – diagonal precision of well k, a vector in $\mathbb{R}^d$ giving the axis-aligned curvature. In code: a.

B_k – low-rank factor of well k, of shape $d \times r$. Supplies the off-diagonal, tilted part of the curvature that a diagonal precision cannot express. In code: B.
r – rank of B_k, i.e. how many oblique directions each well may use. In code: ANISO_RANK, default 4.
P_k – precision matrix of well k, $P_k = \mathrm{diag}(a_k) + B_k B_k^\top$: a diagonal part plus a rank-r correction. The two parts are treated differently by the CfC integrator, which is why they are named separately.
$e_k(h)$ – exponent of well k at hidden state h, the quadratic form $(h-\mu_k)^\top P_k (h-\mu_k)$. In code the two halves appear as diag_term and lr_term.
E_k – energy of well k in the quadratic family, the analogue of e_k for the log-sum-exp construction rather than the sum of bumps.
$g_k(h)$ – **well weight**: $g_k = w_k \exp(-e_k/2)$, the unnormalised strength with which well k is felt at h. In code: bumps. Defined at length, and contrasted with the two quantities below, under "Three weights that are easy to confuse".
r_k – **responsibility** of well k: the *normalised* soft assignment $\mathrm{softmax}_k(-E_k/\tau + \log\pi_k)$, which sums to one over k. Belongs to the quadratic family, not to the production Gaussian potential.
live_frac – **exponent occupancy**: the fraction of token-slots in a bank whose exponent clears an underflow cutoff, so the well still contributes gradient. A count over a population of tokens, not a per-well weight.
$\mathcal{L}$ – PSD low-rank operator $\mathcal{L} = \sum_k g_k B_k B_k^\top$, the aggregate off-diagonal curvature seen by the integrator at one token.
G – factor of that operator, $\mathcal{L} = GG^\top$, formed by concatenating the per-well factors as $G = [\sqrt{g_1}B_1, \ldots, \sqrt{g_K}B_K]$, of shape $d \times Kr$.
$\sigma_{\max}(B_k)$ – largest singular value of the low-rank factor. Its square bounds the stiffest curvature the well contributes, and is the quantity the low-rank cap below bounds.
PR – participation ratio of a well's singular spectrum, $(\sum_i \sigma_i^2)^2 / \sum_i \sigma_i^4$, lying in $[1, r]$. The effective-rank statistic: 1 when the budget sits in one direction, r when the spectrum is flat.
precision_max, precision_lr_max – caps on the diagonal and low-rank curvature channels respectively. The low-rank cap bounds $\|B_k\|_F$ through a $\tanh$ squash and is the one that has been tuned.

Disambiguation: symbols that collide between the two documents

Five symbols carry different meanings in the two deep dives. Neither convention was changed, because each is well established inside its own document; the collisions are listed here instead so that a symbol carried across is not silently misread.

symbol	in the SPLM deep dive	in this document
B	number of well banks	B_k is the low-rank factor of well k, of shape $d \times r$
E	token embedding matrix	E_k is the energy of well k in the quadratic family
L	number of integration steps (depth)	unchanged; the low-rank operator is written $\mathcal{L}$
P	positional embedding matrix	P_k is the precision matrix of well k
V	vocabulary size	V_θ is the potential and V its value at a point

Four of these are disambiguated by the well subscript k: the structured-potential meaning carries one and the SPLM meaning does not, so the subscript is a reliable discriminator wherever it appears.
L is the exception, because the low-rank operator takes no subscript and would otherwise be indistinguishable from the depth. It is therefore written in script throughout this document — $\mathcal{L} = \sum_k g_k B_k B_k^\top$, and $\mathcal{L} = GG^\top$ in

factored form — following the convention this document already uses for the Gaussian $\mathcal{N}$ and for the layer index ℓ. An upright L is always the number of integration steps.
Separately, r is the rank of B_k while r_k is the responsibility of well k — related only by the letter.

Motivation

Potential Wells as Energy Landscapes

Scalar fields and force

A *potential energy function* (or simply *potential*) is a scalar field over a state space:

$V: \mathbb{R}^d \to \mathbb{R}$
At any point x the **force** is the negative gradient of the potential,

$$F(x) = -\nabla V(x),$$

which is the defining property of a **conservative force field**: the work done along any closed path is zero, and the force is completely determined by the scalar potential. This is the structural reason a scalar V is worth learning at all — a single scalar field determines an entire d-dimensional force law, and any force law obtained this way is conservative by construction.
A **potential well** is a local minimum of V. At a minimum x^*,

$$\nabla V(x^*) = 0, \qquad \nabla^2 V(x^*) \succ 0,$$

so the Hessian is positive definite and the force points "downhill" toward x^* from every nearby direction. A particle released near x^* in the presence of friction settles into it. The **depth** of the well determines how strongly the particle is captured; the **curvature** — the spectrum of $\nabla^2 V(x^*)$ — determines how quickly it settles, and, as later sections develop, how stiff the resulting dynamics are for a numerical integrator.
The governing image is a ball rolling on a hilly surface under gravity with friction: position is the state x, height is the potential $V(x)$, the gravitational component along the slope is the force $F = -\nabla V$, friction is the damping γ that dissipates kinetic energy, and equilibrium is the ball at rest at the bottom of a valley. Without friction the ball oscillates indefinitely; with too much friction it barely moves (the overdamped regime); at critical damping it slides into the minimum in the shortest time.
A complex potential over $\mathbb{R}^d$ therefore defines an **energy landscape** — a d-dimensional surface with valleys (wells, the stable attractors), ridges (local maxima, unstable repellers), saddle points (the passes between valleys, which act as transition states), and a global minimum (the ground state). The dynamics of any system governed by V is motion on that landscape, and the remainder of this document is concerned with how such a landscape is parameterised, learned, and integrated.

Semantic Attractor Extraction Algorithm

The Semantic Simulation framework asserts that a trained Scalar Potential Language Model does not just *predict* the next token; it learns a **semantic landscape** – a scalar field $V_\theta(\xi, h)$ over the joint space of contexts ξ and hidden states h – whose **local minima** correspond to coherent semantic configurations.

This is a strong claim. Attention transformers have no analogue of it : their next-token logits are produced by a single matrix multiplication, not by an energy minimization, so they cannot, even in principle, expose a basin structure of "topics the model has internalized". If the SPLM picture is right , we should be able to :

1. Fix a context ξ (e.g. the cumulative mean of an English prefix)
2. start from many random hidden states h_0
3. follow the gradient $-\nabla_h V_\theta(\xi, h)$ to convergence
4. read out the converged h^* through the tied LM head[1]

The following experiment was conducted in order to extract semantic attractors

We use two trained SPLM checkpoints from the Tiny-Shakespeare run with vocabulary = GPT-2 BPE, $d = 128$, mass-mode = `logfreq`.

name	**integrator**	L	Δt	**Val PPL**
Euler L=8	Semi-implicit Euler (SARF-mass)	8	1.0	9.99
Verlet L=16	Stoermer-Verlet (symplectic)	16	0.5	10.43

For each prompt the model is run forward and ξ is captured at the last layer and last-token position. Three hundred and eighty-four seeds h_0 are then built — 128 Gaussian samples matched to the statistics of h_L, 128 random token embeddings, and 128 perturbed points from the real h_L trajectory — and each seed is evolved at **fixed** ξ until convergence. The converged states are clustered with k-means under a silhouette sweep over $K \in [2,10]$, and each cluster centroid is decoded through the tied LM head as $\mathrm{softmax}(c\,E^\top)$. Five prompts spanning narrative, mathematics, science, dialogue, and code supply the context diversity.

Two evolution rules were compared: **gradient descent** (Adam on $V_\theta(\xi, h)$, the literal reading of "find the minima of V_θ") and **damped dynamics** (the model's own semi-implicit integrator on $\ddot{h} = -\nabla V_\theta(\xi, h)/m - \gamma\dot{h}$, the dynamical system actually implemented at inference time).

The two rules disagree, and the disagreement is the result. For a potential that is bounded below the equilibria of the damped second-order system are exactly the critical points of V, so both rules would find the same minima. V_θ **is not bounded below.** Pure Adam descent never converges: across all five prompts and all 384 seeds, $\langle V \rangle$ falls to roughly -2500 by step 300 and -50000 by step 1500 while $\|h\|$ grows from 25 to 2200, and not one of the 384 seeds reaches $\|\nabla V\| < 0.05$. This is structural rather than an artefact of the optimiser: the training loss touches V_θ only through its gradient, so adding a constant to V_θ leaves the loss invariant, and positive rescaling is partially absorbed by the learnable γ, m, and Δt. The absolute scale of V_θ is an unconstrained gauge degree of freedom, which the optimiser is free to drive toward $-\infty$ along any direction where doing so costs nothing.

The damped dynamics, by contrast, **is** prompt-dependently multi-basin. Run for exactly L_{train} steps — the training-time integration depth, beyond which V_θ extrapolates pathologically — the flow concentrates the 384 seeds onto a small number of distinct basins whose decoded token distributions are interpretable, ranging from two to ten basins depending on the prompt, with the Euler $L = 8$ checkpoint at $\Delta t = 1.0$ resolving markedly richer, content-bearing structure than the coarser, punctuation-dominated basins of the Verlet $L = 16$ checkpoint.

The conclusion is therefore more interesting than a simple yes or no, and it fixes the language used in the rest of this document: **"semantic attractor" must be read dynamically, not energetically.** An attractor is a region of h-

[1] In the attractor extraction context, the expression *read out through the **tied** LM head* means computing these logits for the converged h^* and then looking at which tokens have the highest scores – those are the tokens the attractor represents semantically. For details see Appendix.

space onto which the damped flow at fixed ξ concentrates within L_{train} steps. It need not be — and empirically is not — a critical point of V_θ.

Potential Wells and the Concept of Temperature

This section records how a temperature $\tau > 0$ turns a collection of individual wells into a single scalar landscape. The construction used by SQ3 (class `MixtureQuadraticVTheta`) is a temperature-smoothed log-sum-exp mixture of component energies. The rest of the section unpacks that formula: what temperature means, what it does not change about a single well, how the three mixing regimes look geometrically, why the unit value is canonical, and why the same mixing rule applies to well shapes other than quadratics.

Temperature as Boltzmann weight, softmax, and log-sum-exp

The symbol τ that appears in the SQ3 potential is a temperature. The same parameter occurs in statistical mechanics, in classification, and in convex analysis. The three writings look different, but they are the same mathematical object.

• Boltzmann–Gibbs. Given an energy function $E(x)$, the canonical distribution is $p(x) \propto \exp(-E(x)/\tau)$. Small τ concentrates the mass at the energy minimum; large τ flattens the distribution toward uniformity. The normaliser is the partition function $Z(\tau) = \int \exp(-E(x)/\tau)dx$, and the associated free energy is $F(\tau) = -\tau \log Z(\tau)$.

• Softmax. The map $z \mapsto \mathrm{softmax}(z/\tau)$ interpolates between a hard argmax as $\tau \to 0$ and a uniform distribution as $\tau \to \infty$. In a language model this is the difference between greedy decoding and sampling from a nearly flat vocabulary distribution.

• Log-sum-exp. The function $-\tau \log \sum_k \exp(-E_k/\tau)$ is a smooth (soft) minimum of the numbers E_k. The parameter τ is the smoothing radius: as it tends to zero the soft minimum converges to $\min_k E_k$.

In the structured scalar potential of this document the three views are combined into a single landscape. Mixture weights $\pi_k(\xi)$ are a softmax over a linear map of the context ξ (the projection `pi_proj` in `MixtureQuadraticVTheta`). Each well contributes a scalar energy $E_k(\xi, h)$. The SQ3 potential is the temperature-smoothed log-sum-exp of those wells,

$$V_\theta(\xi, h) = -\tau \log \sum_{k=1}^{K} \pi_k(\xi) \exp(-E_k(\xi, h)/\tau).$$

Numerically the same expression is evaluated as $V_\theta = -\tau \,\mathrm{logsumexp}_k(-E_k/\tau + \log \pi_k)$. The implementation then adds a context-dependent offset $b(\xi)$ produced by `b_proj`. That offset does not depend on h, so it shifts the potential without changing the force $f = -\nabla_h V_\theta$.

Temperature here is a property of the **potential**, not of the integrator. The SPLM / PARFLM dynamics are deterministic (Stoermer Verlet / BAOAB, with no Langevin noise). The particle does not sample a Boltzmann distribution at temperature τ; it descends the force field of a landscape whose wells have already been mixed at that temperature. The Boltzmann reading applies at the population level: across many contexts, the converged hidden states are expected to concentrate where e^{-V_θ} is large.

What temperature does not change

The parameter τ does not reshape an individual well. Each component energy E_k is a function of the hidden state h and the context ξ in its own right. For the quadratic family used by SQ1 and SQ3 that function is the diagonal parabola written in the next section,

$$E_k(\xi, h) = \frac{1}{2} a_k(\xi)^\top ((h - \mu_k(\xi)) \odot (h - \mu_k(\xi))),$$

with centre $\mu_k(\xi) \in \mathrm{R}^d$ and positive diagonal precision $a_k(\xi) \in \mathrm{R}^d$, $a_{k,i}(\xi) > 0$. The square on the displacement is the Hadamard (elementwise) square of a length-d vector; the subsequent product with $a_k^\top$ is an ordinary inner product and yields a scalar. In code this is `(a * diff * diff).sum(dim=-1)` with a factor of one-half. Temperature never enters that formula. What τ controls is only the **mixing rule**: how the K parabolas are aggregated into one landscape V_θ.

Three mixing regimes

Write $E_{\min}(\xi, h) = \min_k E_k(\xi, h)$ for the lowest well at the current state, and assume the weights form a probability vector ($\sum_k \pi_k = 1, \pi_k > 0$). The log-sum-exp potential interpolates between a hard minimum, a probabilistic mixture, and a weighted average of the wells.

As $\tau \to 0$ the sum is dominated by the smallest exponent. A standard Laplace argument then gives

$$V_\theta(\xi, h) \longrightarrow \min_k E_k(\xi, h).$$

Geometrically this is a hard Voronoi partition of h-space: at each point only the currently deepest well is visible, and the landscape has sharp creases on the boundaries where two wells exchange the minimiser. The force is then piecewise that of a single parabola. In language-model language the analogue is greedy / argmax decoding: the particle "sees" only one well at a time.

At the canonical value $\tau = 1$ one recovers the negative log-likelihood of a mixture (up to an h-independent constant),

$$V_\theta(\xi, h) = -\log \sum_{k=1}^{K} \pi_k(\xi) e^{-E_k(\xi, h)}.$$

When each E_k is the quadratic energy above, e^{-E_k} is — up to its own Gaussian normaliser — a diagonal Gaussian centred at $\mu_k(\xi)$ with covariance $\mathrm{diag}(1/a_k(\xi))$. Consequently e^{-V_θ} is a Gaussian-mixture density in h given ξ. Nearby wells overlap smoothly: if h sits between two centres, both contribute, and the landscape develops smooth saddles rather than creases.

As $\tau \to \infty$ a first-order expansion of the exponential gives the π-weighted average of the wells. Write $\exp(-E_k/\tau) = 1 - E_k/\tau + O(\tau^{-2})$. Then $\sum_k \pi_k \exp(-E_k/\tau) = 1 - (\sum_k \pi_k E_k)/\tau + O(\tau^{-2})$, and taking $-\tau \log$ of that expansion yields

$$V_\theta(\xi, h) \longrightarrow \sum_{k=1}^{K} \pi_k(\xi) E_k(\xi, h).$$

This is **not** necessarily the uniform average $(1/K)\sum_k E_k$. The uniform average appears only when every weight equals $1/K$. In the generic case every well contributes everywhere, with strength $\pi_k(\xi)$, and the K basins merge into one broad bowl — the analogue of uniform sampling.

A useful intuition is to treat τ as a **blending radius**. At low temperature the landscape is piecewise quadratic and only the nearest (lowest-energy) well matters at a given h. At unit temperature neighbouring wells overlap and the saddles between attractors are smooth. At high temperature the separate attractors dissolve into a single global basin. The three regimes may be summarised as follows. (The later table under "What τ controls" is the same comparison, specialised to the SQ3 discussion in the quadratic-well section.)

τ	**Effective** V_θ	**Geometric picture**
$\tau \to 0$	$\min_k E_k$	Hard Voronoi partition: each well owns its region; sharp creases at basin boundaries.
$\tau = 1$	$-\log \sum_k \pi_k e^{-E_k}$	Smooth soft-minimum. For quadratic E_k, the negative log-likelihood of a Gaussian mixture (up to a constant).
$\tau \to \infty$	$\sum_k \pi_k E_k$	Weighted average of all wells; one merged basin. Reduces to $(1/K)\sum_k E_k$ only when the weights are uniform.

A quantitative sandwich makes the same statement without taking limits. For any $\tau > 0$

$$\min_k E_k(\xi, h) \;\le\; V_\theta(\xi, h) \;\le\; \min_k E_k(\xi, h) - \tau \log \pi_{k_*}(\xi),$$

where k_* is a minimising index. The lower bound holds because a convex combination of Boltzmann weights cannot exceed the largest of them, which is $\exp(-E_{\min}/\tau)$. The upper bound holds because the mixture is at least as large as its largest single term $\pi_{k_*}\exp(-E_{\min}/\tau)$. If the weights are uniform, $\pi_k = 1/K$, the gap is at most $\tau \log K$. Thus V_θ is always a smoothed upper envelope of the hard minimum, and the smoothing width is proportional to temperature.

Unit temperature, responsibilities, and the force

The value $\tau = 1$ is canonical because it matches energy units to probability units with no extra rescaling. Then

$$e^{-V_\theta(\xi,h)} = \sum_{k=1}^{K} \pi_k(\xi) e^{-E_k(\xi,h)},$$

which is, for quadratic wells, the un-normalised Gaussian-mixture density in h given ξ. The minima of V_θ are the modes of that mixture. The force used by the integrator is $f = -\nabla_h V_\theta$, and therefore points toward those modes. Differentiating the log-sum-exp gives an average of the per-well gradients, weighted by the posterior responsibilities

$$q_k(\xi, h) = \frac{\pi_k(\xi)\exp(-E_k(\xi,h)/\tau)}{\sum_{j=1}^{K} \pi_j(\xi)\exp(-E_j(\xi,h)/\tau)}.$$

These are exactly `softmax_k(-E_k/tau + log pi_k)` in `MixtureQuadraticVTheta`. They are nonnegative and sum to one, so they are a soft assignment of the current hidden state to the K wells. For the quadratic family the per-well gradient is the linear spring $a_k(\xi) \odot (h - \mu_k(\xi))$, and

$$\nabla_h V_\theta = \sum_{k=1}^{K} q_k(\xi, h) a_k(\xi) \odot (h - \mu_k(\xi)).$$

The method `analytical_grad` returns this gradient; the integrator uses the force $f = -\nabla_h V_\theta$. In the constructor the default is $\tau = 1$ (`MixtureQuadraticVTheta(..., tau=1.0)`). Making τ a second learnable scalar is redundant for **quadratic** wells: dividing E_k by τ is the same as replacing the precision a_k by a_k/τ, and a_k is already produced by a learned map `a_proj`. For a non-quadratic E_k the two operations are no longer equivalent, and an explicit τ would be a genuine extra degree of freedom.

This log-sum-exp construction is **not** the production Gaussian potential used in Fock-PARFLM. Class `MixtureGaussianVTheta` replaces the mixture of quadratics by a sum of bounded negative Gaussian bumps, precisely because the log-sum-exp force is unbounded. That family has no temperature parameter and is **not** the negative logarithm of a Gaussian mixture; it is treated in the Gaussian Wells subsection below.

The mixing rule does not assume quadratic wells

Nothing in the definition of V_θ requires E_k to be a parabola. The same temperature-smoothed log-sum-exp is an aggregation rule for any collection of K scalar component energies. What changes with the choice of E_k is the shape of each well and the name of the associated Boltzmann weight $e^{-E_k/\tau}$, not the meaning of τ as a blending radius. The three mixing regimes of the previous subsection continue to apply, with the same hard-minimum, unit-temperature, and high-temperature limits.

Form of E_k	**Individual well**	**Boltzmann weight $e^{-E_k/\tau}$**
Quadratic: $\frac{1}{2}a_k^\top((h-\mu_k)\odot(h-\mu_k))$	Axis-aligned parabolic bowl	Diagonal Gaussian
Quartic: $\frac{1}{4}b_k\|h-\mu_k\|_2^4$	Flatter floor, steeper walls	Lighter tails than a Gaussian (the weight decays as a quartic exponential)
Absolute: $c_k\|h-\mu_k\|_2$	Cone (V-shaped)	Laplacian
Bounded bump: $A_k(1-e^{-\kappa_k\|h-\mu_k\|_2^2})$	Saturating well of finite depth	No standard named family
Unconstrained: $\mathrm{MLP}(\xi,h)$	Arbitrary learned surface	Arbitrary

Quadratic wells occupy most of the discussion because, at $\tau=1$, they are exactly a Gaussian mixture, which is the semantic-attractor picture of the paper. The log-sum-exp and the three temperature regimes remain valid for every row of the table. A construction that kept this mixing rule and only changed E_k to a saturating bump would still be a log-sum-exp potential, now with bounded wells. That is **not** what the production Fock-PARFLM code implements. The production module `MixtureGaussianVTheta` is closer in spirit to the saturating row, but it sums the bumps with a minus sign rather than taking $-\tau\log$ of a mixture of Boltzmann weights. The reason, taken up in the Gaussian Wells subsection, is that a log-sum-exp of even bounded wells still produces an unbounded force through the outer logarithm.

Quadratic and Gaussian Potential Wells

<u>Difference between Quadratic and Gaussian Potential Wells</u>
Quadratic and Gaussian potential wells are closely related concepts.

The relationship between them is asymptotic rather than nominal. Expanding a bounded Gaussian well $E(x)=A\left(1-e^{-\kappa\|x-\mu\|^2}\right)$ about its centre gives $E(x)=A\kappa\|x-\mu\|^2+O(\|x-\mu\|^4)$, so near the bottom of the well the Gaussian **is** a quadratic well, with precision $2A\kappa$ and the same linear restoring force. The two forms are indistinguishable to any particle that stays near its attractor, which is the regime the dynamics spends most of its time in.
They part company far from the centre. The quadratic well is unbounded, $E\to\infty$ as $\|x-\mu\|\to\infty$, with a restoring force that grows linearly without limit; the Gaussian well saturates at the finite depth A, and its force decays back to zero, bounded by $\|F\|\le 2A\kappa/e$. That single difference is the entire engineering trade-off between them. An unbounded well guarantees a particle can never escape, at the cost of a force that can grow without bound and destabilise a numerical integrator; a bounded well guarantees a bounded force — and therefore Lipschitz-regular

dynamics compatible with the closed-form integrators developed later — at the cost of allowing a particle with sufficient kinetic energy to escape the basin entirely.

Quadratic Wells

Let us consider a scalar potential well of the form:

$$E_k(\xi, h) = \frac{1}{2} a_k(\xi)^T \big(h - \mu_k(\xi)\big)^2$$

Here the subscript k denotes the k-th well on the semantic energy landscape. Note that this is a quadratic function of h.

This is a conceptual point worth unpacking carefully. The short answer is that "quadratic well" and "Gaussian well" are two names for the same object, seen from two different perspectives.

The equivalence

Consider a single SQ1 component:

$$E_k(\xi, h) = \frac{1}{2} a_k(\xi)^\top (h - \mu_k(\xi))^2$$

This is a quadratic function of h — a parabola in each coordinate. Now consider the corresponding Boltzmann weight:

$$p_k(h \mid \xi) \propto e^{-E_k(\xi,h)} = \exp\left(-\frac{1}{2} a_k^\top (h - \mu_k)^2\right)$$

That exponential of a negative quadratic is exactly a (diagonal) Gaussian with mean $\mu_k(\xi)$ and precision $a_k(\xi)$. So:

• From the dynamics side: the potential is a quadratic well (parabolic energy surface).

• From the probabilistic side: the same object is a Gaussian well (the stationary distribution of the damped flow is Gaussian).

The two phrases describe the same mathematical shape — one in energy language, one in probability language.

Why SQ3 recovers the Gaussian mixture

At temperature $\tau = 1$, the SQ3 potential is:

$$V_\theta = -\log \sum_{k=1}^{K} \pi_k(\xi) e^{-E_k(\xi,h)}$$

Rearranging:

$$e^{-V_\theta} = \sum_{k=1}^{K} \pi_k(\xi) e^{-E_k(\xi,h)}$$

So e^{-V_θ} is proportional to the density of a Gaussian mixture model. Equivalently, $V_\theta = -\log p(h \mid \xi)$ up to a constant, which is the negative log-likelihood of that GMM. The minima of V_θ are the modes of the mixture — the Gaussian well centres $\mu_k(\xi)$.

The parameter τ (temperature) in the quadratic well design

What τ controls

τ	Behaviour of V_θ
$\tau \to 0$	$V_\theta \to \min_k E_k$ — hard assignment to deepest basin; sharp, non-smooth
$\tau = 1$	V_θ = −log Σ_k π_k e^{−E_k} — exact negative GMM log-likelihood
$\tau \to \infty$	$V_\theta \to \sum_k \pi_k E_k$ — weighted average of the wells (equals $(1/K)\sum_k E_k$ only for uniform π); one merged basin.

$\tau = 1$ is the canonical choice because it connects SQ3 directly to the Gaussian mixture motivation of Section 4 of the paper [1], as derived in Potential Wells and the Concept of Temperature above: energy units and probability units match, and e^{-V_θ} is the mixture density. The three mixing regimes and the sandwich bound are not repeated here.

The practical punchline

The SPLM paper motivates V_θ as governing a landscape of Gaussian wells (semantic attractor basins). SQ3 is that landscape, by construction. You are not approximating Gaussian wells with quadratics — you are writing them down directly in their exact mathematical form, then mixing them with a log-sum-exp (which is the soft generalisation of the minimum over the K components). The "quadratic" and "Gaussian" labels are just two ways to say the same thing.

The labels SQ1 / SQ3 / SQ5 refer to the structured V_θ sweep from Section 17 of [2] (Semantic_Attractor_Extraction.md).

Label	V_θ form
SQ1	K=1 diagonal quadratic well (planned, not yet run)
SQ3	K=4 mixture of diagonal quadratic wells (the main result)
SQ5	Unconstrained MLP reference

A single SQ1 component (subscripted by k) is one diagonal quadratic energy well:

$$E_k(\xi, h) = \frac{1}{2} a_k(\xi)^\top (h - \mu_k(\xi))^2$$

where $\mu_k(\xi) \in R^d$ is the context-dependent centre of the well (where the potential is minimised), and $a_k(\xi) \in R^d_{>0}$ is the context-dependent diagonal precision (sharpness / stiffness along each coordinate). Both μ_k and a_k are produced by small MLPs that take the EMA context vector ξ as input.

This quadratic energy is exactly the negative log of a diagonal Gaussian. The Boltzmann weight

$$p_k(h \mid \xi) \propto e^{-E_k(\xi,h)} = \exp\left(-\frac{1}{2} a_k^\top (h - \mu_k)^2\right)$$

is a Gaussian with mean $\mu_k(\xi)$ and diagonal covariance $\mathrm{diag}(1/a_k(\xi))$. In dynamics language this is a parabolic potential well that attracts hidden states toward $\mu_k(\xi)$ with stiffness $a_k(\xi)$; in probability language it is a Gaussian distribution over h conditioned on ξ.

SQ3 achieved a perplexity of 184.5, matching the MLP baseline. It is defined as a log-sum-exp of $K = 4$ such components (`MixtureQuadraticVTheta`) and is the negative log-density of a mixture of K Gaussian wells at $\tau = 1$. That is **not** the production bump potential `MixtureGaussianVTheta` used in the fock_gaussian_sarf Phase 5 run; the two share the same per-well energy E_k but combine the wells differently (see the next subsection).

Gaussian Wells

The previous subsection derived the well energy E_k purely as a quadratic form. The production `V_theta` modules used across the Fock-PARFLM / SPLM codebase (`model_gaussian_vtheta.py`) do not evaluate the log-sum-exp mixture potential of the previous subsection directly. Instead they implement the corresponding Gaussian-well

potential as a sum of structurally bounded negative Gaussian bumps, replacing SQ3 for exactly this reason: the log-sum-exp mixture's force is unbounded, while a sum of bumps has both potential and force bounded by construction. Two variants of this bump potential exist side by side in the codebase — an isotropic variant, in which the precision a_k is diagonal (every well is an axis-aligned ellipsoid), and an anisotropic variant, in which a learned low-rank correction is added to the diagonal precision so that each well can tilt into a non-axis-aligned ellipsoid. Both are given below with their closed-form gradients and the code that implements them.

Isotropic implementation: MixtureGaussianVTheta

The isotropic well bank keeps the diagonal precision a_k of the quadratic well defined above, and turns it into a bounded potential by summing negative Gaussian bumps rather than taking a log-sum-exp:

$$V_\theta^{\text{iso}}(\xi, h) = -\sum_{k=1}^{K} w_k(\xi)\exp\big(-E_k(\xi, h)\big)$$

where $w_k(\xi)$ are softmax mixture weights and $E_k(\xi, h)$ is the same diagonal quadratic energy defined above. Because every bump $w_k(\xi)\exp(-E_k)$ is non-negative and bounded above by $w_k(\xi)$, the total potential is bounded by construction, unlike the unbounded-force log-sum-exp mixture of the previous subsection. Unlike SQ3, this bump potential is **not** the negative log of a GMM: $e^{-V_\theta^{\text{iso}}} = \exp(\sum_k w_k e^{-E_k})$ is not a mixture density. Differentiating term-by-term gives the per-well gradient of V (not the force):

$$\nabla_h V_k^{\text{iso}}(\xi, h) = w_k(\xi)(a_k(\xi) \odot (h - \mu_k(\xi)))\exp(-E_k(\xi, h))$$

summed over $k = 1, \dots, K$ for the total gradient $\nabla_h V_\theta^{\text{iso}}$. The force used by the integrator is the opposite, $f = -\nabla_h V_\theta^{\text{iso}}$ (in code, `f_theta = -analytical_grad(...)`). This gradient is exactly class `MixtureGaussianVTheta` in model_gaussian_vtheta.py (repository path notebooks/conservative_arch/parf/model_gaussian_vtheta.py):

model_gaussian_vtheta.py, class `MixtureGaussianVTheta`, lines 122–154:

```
    def _components(self, xi: torch.Tensor):
        lead = xi.shape[:-1]
        mu = self.mu_proj(xi).view(*lead, self.K, self.d)
        a = (F.softplus(self.a_proj(xi)) + 1e-4).view(*lead, self.K, self.d)
        w = F.softmax(self.w_proj(xi), dim=-1) * self.w_scale   # (..., K)
        return mu, a, w

    def forward(self, xi: torch.Tensor, h: torch.Tensor) -> torch.Tensor:
        mu, a, w = self._components(xi)
        h_e = h.unsqueeze(-2)
        diff = h_e - mu
        exponent = -0.5 * (a * diff * diff).sum(dim=-1)
        bumps = w * torch.exp(exponent)
        V = -bumps.sum(dim=-1, keepdim=True)
        return V

    def analytical_grad(self, xi, h):
        mu, a, w = self._components(xi)
        h_e = h.unsqueeze(-2)
        diff = h_e - mu
        exponent = -0.5 * (a * diff * diff).sum(dim=-1)
```

```
        g = w * torch.exp(exponent)
        per_comp = a * diff * g.unsqueeze(-1)
        return per_comp.sum(dim=-2)
```

Reading the code against the equations above: `a_proj` produces the diagonal precision $a_k(\xi)$ (softplus keeps it positive), `exponent` is $-E_k(\xi, h)$, `bumps` is the summand $w_k(\xi)\exp(-E_k)$, and `per_comp` before the final sum is exactly the per-well gradient term $a_k(\xi) \odot \big(h - \mu_k(\xi)\big)g$ derived above (the code omits the optional `precision_max` and `force_norm_max` clamps for brevity; both are stability safeguards, not part of the closed-form gradient itself).

Anisotropic implementation: diagonal + low-rank precision

The isotropic well's precision a_k is diagonal, so every well is an axis-aligned ellipsoid in h-space. `AnisotropicMixtureGaussianVTheta` (used in the Fock-PARFLM scale-up sweeps) replaces the diagonal precision with a diagonal-plus-low-rank correction:

$$P_k(\xi) = \Sigma_k(\xi)^{-1} = \mathrm{diag}(a_k(\xi)) + B_k(\xi)B_k(\xi)^{\top}, B_k(\xi) \in \mathrm{R}^{d\times r}$$

with rank r (`r=4` by default — `ANISO_RANK` in the implementation). The $B_k{B_k}^{\top}$ term lets each well tilt into a non-axis-aligned ellipsoid; its energy is

$$E_k^{\mathrm{aniso}}(\xi, h) = \frac{1}{2}(h - \mu_k)^{\top}(\mathrm{diag}(a_k) + B_k B_k^{\top})(h - \mu_k) = \frac{1}{2}[a_k^{\top}((h - \mu_k) \odot (h - \mu_k)) + \|B_k^{\top}(h - \mu_k)\|_2^2]$$

and the bounded bump potential and gradient take exactly the same form as the isotropic case, with E_k replaced by E_k^{aniso}:

$${V_\theta}^{\mathrm{aniso}}(\xi, h) = -\sum_{k=1}^{K} w_k(\xi)\exp\left(-E_k^{\mathrm{aniso}}(\xi, h)\right)$$

$$\nabla_h V_k^{\mathrm{aniso}}(\xi, h) = w_k(\xi)(a_k(\xi) \odot (h - \mu_k(\xi)) + B_k(\xi)(B_k(\xi)^{\top}(h - \mu_k(\xi))))\exp(-E_k^{\mathrm{aniso}}(\xi, h))$$

The additional $B_k(\xi)(B_k(\xi)^{\top}(h - \mu_k(\xi)))$ term is the low-rank piece of $\nabla_h E_k^{\mathrm{aniso}}$, hence of the gradient $\nabla_h V_k^{\mathrm{aniso}}$; it vanishes when $B_k = 0$, recovering the isotropic gradient above term-for-term. The force is again $f = -\nabla_h V$. Implementation (colab_fock_aniso_gaussian_fockreg_tinystories.ipynb, Cell 4: "Anisotropic Gaussian V_theta"):

```
    def forward(self, xi, h):
        mu, a, w, B = self._components(xi)
        h_e = h.unsqueeze(-2)
        diff = h_e - mu

        diag_term = (a * diff * diff).sum(dim=-1)
        Bt_diff = torch.einsum('...kd,...kdr->...kr', diff, B)
        lr_term = (Bt_diff * Bt_diff).sum(dim=-1)

        exponent = -0.5 * (diag_term + lr_term)
        bumps = w * torch.exp(exponent)
        return -bumps.sum(dim=-1, keepdim=True)

    def analytical_grad(self, xi, h):
        mu, a, w, B = self._components(xi)
```

```
h_e = h.unsqueeze(-2)
diff = h_e - mu

diag_term = (a * diff * diff).sum(dim=-1)
Bt_diff = torch.einsum('...kd,...kdr->...kr', diff, B)
lr_term = (Bt_diff * Bt_diff).sum(dim=-1)

exponent = -0.5 * (diag_term + lr_term)
g = w * torch.exp(exponent)

grad_diag = a * diff
grad_lr = torch.einsum('...kdr,...kr->...kd', B, Bt_diff)
per_comp = (grad_diag + grad_lr) * g.unsqueeze(-1)
return per_comp.sum(dim=-2)
```

Here `diag_term` is $a_k^\top((h-\mu_k)\odot(h-\mu_k))$, which is **twice** the diagonal part of E_k^{aniso} (the factor $\frac{1}{2}$ sits in `exponent = -0.5 * (...)`). `lr_term` is the Euclidean square $\|B_k^\top \text{diff}\|_2^2$, **not** a Hadamard square of an r-vector left unsummed and is twice the low-rank part of the energy. `grad_lr` implements $B_k(B_k^\top \text{diff})$, the low-rank piece of $\nabla_h E$ and therefore of $\nabla_h V$. The base `_components` method is identical to the isotropic bank's, extended with one more learned projection, `B_proj`: in_d → K × d × r, that produces $B_k(\xi)$.

Empirical comparison

On a controlled TinyStories run ($d = 256$, identical routing, gradient clipping, $\gamma = 0.3$, WSD schedule, same seed), the isotropic Gaussian `V_theta` trails the unconstrained MLP baseline, but the anisotropic variant with Fock coupling regularisation closes that gap and reverses it:

V_theta variant	**Best val PPL**	**Notes**
MLP (ScalarPotentialMultiXi)	9.70	Unconstrained; unbounded force
Isotropic Gaussian (MixtureGaussianVTheta)	~13 (projected)	Diagonal precision; bounded force
Anisotropic Gaussian + Fock-reg	9.14	Diagonal + rank-4 precision; bounded force

The anisotropic well recovers the directional structure (ridges and saddles between wells) that the purely diagonal, axis-aligned isotropic well cannot represent, while retaining the same closed-form gradient, bounded force, and Lipschitz regularity the isotropic variant was introduced for — the properties the CfC propagator, BAOAB integrator, and Langevin O-step upgrades all require. The structural constraint of a Gaussian `V_theta` is therefore not a strictly weaker choice than the unconstrained MLP: with sufficient force-landscape complexity (anisotropy) and the Fock coupling regulariser that keeps the mechanism engaged, it is the stronger model on raw perplexity as well as the only one compatible with the closed-form integrator upgrades.

The isotropic and anisotropic well designs above generalise along two independent axes that the remainder of this section develops in turn: how much curvature budget an individual well should be given (rank selection), and how the well should read a context built from more than one timescale (multi-channel and depth-conditioned context banks).

Three weights that are easy to confuse

Three distinct quantities in this codebase all answer some version of "how much is well k involved here", and they are not interchangeable. Two are per-well weights that differ in whether they are normalised; the third is not a per-

well weight at all. Conflating them has already produced at least one wrong inference, so they are pinned down here.

1. The well weight g_k — unnormalised, and what the production potential actually uses. Writing the exponent of well k at hidden state h as

$$e_k(h) = \sum_j a_{k,j}\left(h_j - \mu_{k,j}\right)^2 + \|B_k^\top(h-\mu_k)\|^2 = (h-\mu_k)^\top P_k (h-\mu_k),$$

with $P_k = \mathrm{diag}(a_k) + B_k B_k^\top$ as above, the well weight is

$$g_k(h) = w_k \exp(-1/2\, e_k(h)).$$

In the anisotropic implementation code above this is bumps, the product of the mixture weight w with exp(exponent). It carries three distinct jobs, which is why it is worth a name:

1. it *is* the well's contribution to the potential, since $V_\theta = -\sum_k g_k$;
2. it is the coefficient with which the well's curvature enters the frozen harmonic split used by the CfC integrator — the diagonal spring is $k_{\mathrm{diag}} = \sum_k g_k\, a_k$ and the low-rank operator is $\mathcal{L} = \sum_k g_k\, B_k B_k^\top$, assembled from the factor $G = [\sqrt{g_1}B_1, \ldots, \sqrt{g_K}B_K]$;
3. it scales the well's contribution to the analytic force.

Crucially g_k is **not normalised**. It does not sum to one over k; it lies in $(0, w_k]$ and is free to be small for every well at once, which is exactly what happens at a position far from the whole bank. A token where all wells are numerically dead is a real and expected state, not a degenerate one.

2. The responsibility r_k — normalised, and belonging to a different family. The log-sum-exp construction discussed earlier in this document produces $r_k = \mathrm{softmax}_k(-E_k/\tau + \log\pi_k)$, which is nonnegative and sums to one, so it is a genuine soft assignment of h among the wells. That normalisation is the whole difference. A responsibility says *which* well owns this position; a well weight says *how strongly* a particular well is felt there, with no reference to any other well.

The two also belong to different potentials — responsibilities to the mixture of quadratics, well weights to the sum-of-bumps Gaussian potential that Fock-PARFLM actually trains. Statements about one do not transfer.

3. Exponent occupancy — a count over tokens, not a weight. Sometimes called live_frac, this is the fraction of token-slots in a bank whose exponent exceeds a cutoff (in practice e_k small enough that $\exp(-e_k/2) \gtrsim 5 \times 10^{-5}$, so the well still contributes gradient rather than having underflowed). It measures **how many** wells are live across a batch, not how strongly any one of them is weighted. It is a diagnostic over a population of tokens; g_k is a value at a single token.

Why the distinction has teeth: truncation moves g_k upward

The low-rank factor appears inside the exponent, so any intervention on B_k changes the well weight as a side effect — and the direction is not the intuitive one.

Replacing B by its rank-r' truncation means $B \mapsto BP$ for an orthogonal projector $P = V_{r'} V_{r'}^\top$. Since projectors are norm-nonincreasing,

$$\|(BP)^\top x\|^2 = \|PB^\top x\|^2 \le \|B^\top x\|^2$$

for every x. So truncation weakly **lowers** the exponent and therefore weakly **raises** the well weight, at every well and every token, with equality only when $B^\top x$ already lies in the retained subspace. This is deterministic and pointwise, not a statement about averages.

The consequence is that **a rank truncation is not a clean reduction of curvature.** It removes curvature from each well while simultaneously making every well fire harder, and the two effects oppose each other in anything downstream that consumes g_k — including $\mathcal{L} = \sum_k g_k\, B_k B_k^\top$, whose largest eigenvalue has been measured to *increase* under truncation. Any experiment that truncates B is therefore varying two things at once, and which one dominates is an empirical question rather than something to be assumed. The rank-selection procedure in the next section inherits this caveat directly.

The contrasting control — replacing B by $B + N$ for isotropic noise N carrying exactly the Frobenius energy the truncation discards — moves the exponent the other way, but only in expectation:

$$\mathbb{E}\|(B+N)^\top x\|^2 = \|B^\top x\|^2 + \mathbb{E}\|N^\top x\|^2 \geq \|B^\top x\|^2,$$

since the cross term $2\langle B^\top x, N^\top x\rangle$ vanishes in expectation but not realisation-by-realisation. Measured over 4000 synthetic wells at the deployed shape, the two perturbations shift the mean exponent by equal and opposite amounts to within about 2% — but while truncation moved every single well in the same direction, **roughly a third of wells moved the "wrong" way under matched noise**. A comparison between the two is therefore a statement about the distribution of g_k, not a per-well sign flip, and an effect driven by a few extreme wells could hide in that third.

Curvature Diagnostics and Rank Selection for the Anisotropic Well

The anisotropic precision $P_k = \mathrm{diag}(a_k) + B_k B_k^\top$ introduced above splits the well's curvature into two channels that are qualitatively different objects, and — critically — that the CfC-BAOAB integrator treats differently. Expanding the quadratic form makes the split explicit:

$$(h-\mu_k)^\top P_k (h-\mu_k) = \sum_j a_{k,j}\,(h_j - \mu_{k,j})^2 + \|B_k^\top (h-\mu_k)\|^2.$$

The diagonal term is axis-aligned and is integrated exactly by the closed-form harmonic propagator; the low-rank term is oblique and enters the integrator as an explicit kick, which is stable only while $\omega\Delta t < 2$ for the effective frequency ω it induces. In deployed scale-up runs this asymmetry is large: the diagonal precision is capped at $2/d$ (a stability constraint inherited from an earlier, unconditionally-Verlet integrator, and no longer load-bearing now that the diagonal channel is integrated exactly), while the low-rank channel is capped independently by a separate budget, precision_lr_max, and in practice carries essentially the entire exponent. The scalar that actually controls the well's sharpness and the integrator's stability margin is therefore not the nominal rank r but the top squared singular value of the low-rank factor,

$$s_k \equiv \sigma_{\max}(B_k)^2 \leq \|B_k\|_F^2 = \mathrm{tr}(B_k B_k^\top).$$

Along the sharpest direction the peak force scales as $\sqrt{s_k}$ and the worst-case parameter gradient scales as s_k itself, which is why s_k — not the rank that bounds it — is the quantity worth measuring and bounding directly.

Effective rank as a measurement: the participation ratio

Rank r is a design choice — how many columns B_k is allowed — but it says nothing about how the well actually spends that budget. For a factor B_k with descending singular values σ_i, the *participation ratio*

$$\mathrm{PR}(B_k) = \frac{(\sum_i \sigma_i^2)^2}{\sum_i \sigma_i^4} \in [1, r]$$

is the standard effective-rank statistic — the inverse participation ratio of the normalised eigenvalue distribution of $B_k B_k^\top$ — with exactly the two limits the rank question needs: $\mathrm{PR} = 1$ when the whole budget sits in one direction, $\mathrm{PR} = r$ when the spectrum is flat. Measured against a rank r, the reading is: $\mathrm{PR} \geq 0.75r$ means the budget is saturated and a higher rank has a genuine case; $\mathrm{PR} \leq 0.5r$ means the budget is not being used and the extra rank is idle parameters; the band between is ambiguous and should be weighed against the cost analysis below.

Correction (September 2026): those bands compare PR to the wrong reference, and the measurement below supersedes them. Reading $\mathrm{PR} \geq 0.75r$ as "saturated" assumes that $\mathrm{PR} \to r$ means a budget fully used. It does not. A random $d \times r$ Gaussian already has a near-flat spectrum, so an *untrained* model scores near the ceiling. Measured on this architecture at $r = 4$, initialisation gives $\mathrm{PR} = 3.96$; the trained model gives 3.69. Training moves the statistic **down** from that null, and the meaningful quantity is the concentration $\mathrm{PR}_0 - \mathrm{PR}$, not PR itself.

Two further cautions come from the same measurement. First, PR alone cannot separate "flat because every direction is in use" from "flat because the well is doing nothing" — that needs $\|B_k\|_F$ read alongside it, since only a well at the cap is spending its full budget. Second, the statistic must be read per (layer, channel) site rather than pooled: pooled quantiles average over sites and conceal precisely the structure the rank decision turns on.

Measured across all 40 sites at step 96410, every site sits at the cap ($\|B_k\|_F = 1.000$), so the spread in PR is pure redistribution rather than some channels lying idle — and that spread is carried by **channel** (means 3.18, 3.74, 3.82, 3.64, 3.61; range 0.65) roughly twice as strongly as by layer (range 0.33). A single global rank therefore serves the channels unequally, which is the argument for a per-channel allocation.

A second, independent use of the same statistic follows directly from the cap. Under a binding Frobenius cap, $\sigma_{\max}^2$ can only grow by *concentration* — by the well moving mass from many directions into one. Since spike magnitude scales as $\sigma_{\max}(B_k)^2$, this predicts that a curvature-led gradient spike may literally be a moment of spectral collapse: comparing the participation-ratio spectrum at a healthy checkpoint against the spectrum at a spike checkpoint tests that directly, with the same statistic and no additional machinery.

Rank under a binding cap: redistribution, not addition

The bounded low-rank factor used in production caps the Frobenius norm with a tanh squash, $\|B_k\|_F \leftarrow b\tanh(\|B_k^{\text{raw}}\|_F / b)$, where $b = \sqrt{\text{precision_lr_max}}$. Once the cap saturates — which it does whenever the raw, uncapped norm is well above b — the total curvature $\sum_i \sigma_i^2$ is pinned at b^2 *regardless of rank*. Increasing r does not add curvature; it changes only the range of distributions the fixed budget can take:

$$\frac{b^2}{r} \leq \sigma_{\max}(B_k)^2 \leq b^2,$$

with the lower bound attained by a perfectly flat spectrum and the upper bound by a fully degenerate one. This produces a genuinely counterintuitive consequence, and it is the reason rank is worth measuring rather than assumed: under a binding cap, **higher rank can make the well safer, not more dangerous** — but only if the learned spectrum actually spreads across the available directions. Doubling the rank from 4 to 8 with a flat spectrum halves $\sigma_{\max}^2$ at identical total curvature; if instead the well drives its whole budget into a single direction, rank 8 is indistinguishable from rank 4 — or from rank 1 — on this axis, and the extra parameters buy nothing. Rank is an upper bound on how many directions a well *may* use, never a guarantee that it uses them; the participation ratio is precisely what distinguishes the two.

That distinction has real weight behind it, because the low-rank factor is not a small piece of the model. Instantiating the modules directly shows the projection B_{proj} alone accounts for roughly two-thirds of V_θ's parameters at rank 4, and V_θ itself is close to half of the whole network — so the low-rank factor is on the order of thirty percent of every parameter in the model. Raising the rank from 4 to 8 is correspondingly close to a one-third increase in total model size. That is a real opportunity cost against spending the same budget on depth or width, which is why the rank question deserves measurement before it is answered by convention.

A procedure for choosing the rank

The rank is naturally a single scalar shared by every well in every bank at every layer, but the participation ratio is a per-(layer, channel, well) quantity, and there is no reason for that distribution to be tight — different layers and different context channels routinely carry very different amounts of salience. The honest target is therefore not "the optimal rank" in the abstract but the optimal rank *allocation*, of which a single global r is the degenerate, currently-implemented special case. A staged, cost-ordered procedure for approaching that target:

Stage 0 — precondition. Confirm the Frobenius norm at the median well is close to $\sqrt{\text{precision_lr_max}}$, i.e. that the cap is actually binding. If it is not, the redistribution argument above does not apply and the rank question reduces to an ordinary capacity question instead.

Stage 1 — measure the effective rank. Read the participation-ratio distribution across wells directly from existing checkpoints, at no additional training cost, since the cap being binding constrains only the *total* curvature and says nothing about how it distributes — so the measurement is valid under the deployed configuration without needing to relax anything. If the median participation ratio sits at or below $r/2$, the model is not using the rank it already has, and additional rank will not help; the available levers are a flatness incentive — a penalty on the ratio of $\sigma_{\max}^2$ to

$\|B_k\|_F^2$ that pushes the existing budget to spread across directions — or an outright reduction in rank to reclaim parameters.

Stage 2 — rank-truncation ablation. The participation ratio is a purely geometric statistic; it does not by itself establish that the small singular directions are functionally irrelevant, since a low participation ratio could coexist with tail directions that happen to align with something the loss cares about. The direct test replays a captured batch with the realised B_k truncated to its rank-r' SVD for $r' < r$, holding everything else fixed, and reports the relative force error $\|f_{\text{trunc}} - f_{\text{full}}\|/\|f_{\text{full}}\|$ alongside the resulting loss and gradient norm. If truncating rank 4 to rank 2 changes the force by only a fraction of a percent, the top two directions are carrying the well's entire signal and the remaining parameters are dead weight. This ablation is asymmetric by construction: truncation can only test downward, since directions beyond the trained rank were never learned and cannot be recovered by inspection.

Stage 3 — a short pilot at higher rank. Only when stage 1 shows saturation *and* stage 2 shows the tail directions carry real force is the rank genuinely binding, in which case the only way to learn what more rank buys is to train at it — a short pilot, re-measuring the participation ratio afterward. Saturation again near the new rank means the data wants still more; a plateau partway to the new rank identifies the knee as the answer.

Stage 4 — per-well allocation. If the measured distribution is heterogeneous across wells and layers rather than concentrated, a single global rank is simultaneously over-serving some wells and starving others, and the more precise fix is architectural: a per-bank rank (cheap, since the low-rank projection is already organised per bank, and layer salience gives a principled prior for which banks deserve more), or a shared factor of generous rank with a per-well soft gate that lets each well draw only what it needs.

Note on multi-ξ context channels and K-EMA

Role of ξ in the Euler-Lagrange step

At each integration layer the hidden state $h_t \in \mathbb{R}^d$ of token t is driven by a conservative force from a scalar potential that is allowed to depend on a causal context summary ξ_t:

$$\mathfrak{m}_t \ddot{h}_t + \gamma \dot{h}_t = -\nabla_{h_t} V_\theta(\xi_t, h_t) \quad \text{(a1.ode)}$$

(In the related models PARFLM, Fock-PARFLM there are additional pair and Fock terms which do not replace ξ.)

Without ξ, $V_\theta(\,\cdot\,, h_t)$ would be a *memoryless* well: token t would feel only its own current state. ξ is the mechanism by which *past hidden states* enter the single-particle potential while remaining *a fixed external field* for the h_t-gradient (so the force stays causal and conservative in h_t).

In the SARF-faithful stack, ξ is recomputed at every layer from the current h-stream, not frozen from layer-0 embeddings. Implementation: `causal_force=True` feeds `h.detach()` into the ξ block so autograd cannot route future V_θ values back into past tokens (that is the anti-causal leak that collapsed pre-fix multi-ξ PPL).

Single channel : rank-1 causal mean

The original SPLM / SARF construction uses one context vector, the causal cumulative mean

$$\xi_t = \frac{1}{t}\sum_{s \le t} h_s \quad \text{(ccm.1)}$$

(`causal_cumulative_mean` in `sarf_variant/model_sarf.py`)

Then,

$V_\theta : \mathbb{R}^{2d} \to \mathbb{R}$, input $= \mathrm{cat}(\xi_t, h_t)$

This is a *rank-1 summary* of the prefix : any two prefixes with the same arithmetic mean of hidden states are indistinguishable to V_θ. Attention, by contrast, mixes the prefix in a content-conditioned way. The single-ξ bottleneck (demonstrated by experiment E11/R6 ladder available in the companion repo) is the structural reason the multi-channel extension exists.

K-EMA : K causal weighted averages at several horizons

Multi-ξ replaces the one mean with K parallel exponential kernels (`MultiChannelXi` in `multixi/model_multixi.py`). Channel k has a decay $\alpha_k \in (0,1)$:

$$W_k[t,s] = \frac{\alpha_k^{t-s}}{\sum_{r \le t} \alpha_k^{t-r}} \ (s \le t) \ , \quad W_k[t,s] = 0 \ \ (s > t) \ , \ \ \xi_t^{(k)} = \sum_{s \le t} W_k[t,s] h_s \quad \text{(chk.1)}$$

Weights in (chk.1) are built with in log-space and are row-normalized with `logsumexp` , so each row sums to 1 even for α near 0 or 1.

Endpoint interpolation (this is the implemented kernel, not the classical IIR EMA):

$\alpha_k \to 0 : \ W_k = I\bullet$, so $\xi_t^{(k)} = h_t$ (no past);
$\alpha_k \to 1 : \ W_k\bullet$ is the causal uniform mean , so $\xi_t^{(k)}$ recovers the single-ξ baseline;
$\alpha_k \in (0+\varepsilon, 1-\varepsilon)\bullet$ where $\varepsilon > 0$ is small enough but fixed : a causal weighted mean with effective horizon $\tau_k \sim 1/(1-\alpha_k)$.

The architecture therefore is strict superset : driving $\alpha_k \to 1$ recovers the old context; other channels can keep shorter or longer memory.
Note that this interpolation (chk.1) is not the same as the textbook recurrence

$$\xi_t = \alpha \xi_{t-1} + (1-\alpha) h_t \quad \text{(iir.1)}$$

That IIR form is unnormalized and its limit $\alpha \to 1$ limit freezes early tokens rather than recovering the cumulative mean. In [1], (iir.1) is denoted as `R6.h.0`. With (iir.1) when $\alpha_k \to 0$ $\xi_t^{(k)} = h_t$ like with (chk.1), however when $\alpha_k \to 1$ $\xi_t^{(k)} \approx \xi_{t-1}^{(k)}$ (memory freezes).

The code in the companion repo uses the row-normalized kernel (chk.1) above with $\alpha_k = \mathrm{sigmoid}(\mathrm{raw}_k)$ which guarantees $\alpha_k \in (0,1)$. The decays α_k can be frozen or learnable (`xi_learnable`). All current models in the SPLM family use learnable decays.

Default Configuration : $K = 4$ (config field `xi_channels`)

Log-spacing is the prior that relevant linguistic timescales are scale-free up to a paragraph length: horizons sit on a geometric grid rather than on an ad-hoc list.

Forward shape: $h \in \mathbb{R}^{B \times T \times d} \longmapsto \xi \in \mathbb{R}^{B \times T \times d \times k}$.

Note on the hidden width of the MLP V_θ

This note is on the difference between the hidden width of MLP V_θ denoted with d_V and the number of hidden dimensions of the embeddings (or state) d. d_V and d represent different widths for two different objects.
The hidden state dimensions width d represents the fact that each semantic particle (vector embedding of a token) lives in $\mathbb{R}^d$ (the current TinyStories scale-up model code uses $d = 256$; some of the previous experiments used $d = 192$ and $d_V = 768$). d_V is the MLP width of the scalar potential $V_\theta(\xi, h)$. That network is not a residual stream and it maps a concatenated input to one energy:

```
    layers: List[nn.Module] = [nn.Linear(2 * d, hidden), nn.GELU()]
    ...
    layers += [nn.Linear(hidden, 1)]
def forward(self, xi: torch.Tensor, h: torch.Tensor) -> torch.Tensor:
    return self.net(torch.cat([xi, h], dim=-1))
```

So the first layer already sees the concatenated pair (ξ, h) of dimension $2d$ in the single-context `ScalarPotential`, and dimension $(K_\xi + 1)d$ once the K-EMA bank is concatenated with h in `ScalarPotentialMultiXi`. The quantity d_V (`v_hidden`) is the width of the GELU stack that follows that first map, not the embedding width d.
The first linear map of `ScalarPotential` therefore has shape $2d \to d_V$. In the TinyStories scale-up this is a map $512 \to 1024$ when $d = 256$ and $d_V = 1024$. That is only a factor of two above the concatenated input, which is an ordinary expansion for a feed-forward network, not an accidentally oversized hidden layer. The multi-channel module `ScalarPotentialMultiXi` widens only that first map, to $(K_\xi + 1)d \to d_V$, and leaves the rest of the GELU stack at width d_V. How the K-EMA bank is consumed is taken up in the next subsection; the point of the present note is why d_V itself is chosen larger than d.
The scalar potential is asked to define a complicated landscape on R^d: many wells, saddles, and a context-dependent force $f = -\nabla_h V_\theta$. A network that mapped $2d$ (or $(K_\xi + 1)d$) down to a hidden width of order d and then to a single energy would be a severe bottleneck. Universal-approximation heuristics for a single hidden layer say that the hidden width should be at least the input dimension if the target function is allowed to be even moderately oscillatory. Here the target is worse than a generic function of h: it must also change with ξ, and the integrator trains through the gradient $\nabla_h V_\theta$ rather than through the scalar value alone. If d_V is comparable to d, that gradient tends to be too smooth, in effect low-rank as a map of h, and the force cannot steer particles into distinct semantic basins.
The same expansion appears in a Transformer for a different reason. Self-attention works at width d, but the position-wise feed-forward sublayer typically expands to $4d$ before writing back. In the SPLM family V_θ is the analogue of that feed-forward: it is the learned nonlinearity that shapes the trajectory of h. Setting d_V to about $3d$ or $4d$ is therefore following an established capacity pattern, not matching the embedding width for a misplaced sense of fairness. Typical pairings in this repository are:
So $d_V > d$ is the expected operating point. Setting $d_V = d$ is not a parameter-matched control of the landscape; it usually underfits the potential, and the force that the integrator sees is too impoverished to be compared with a $4d$ run on equal terms. The Helmholtz H1.5 cells that narrowed `v_hidden` did so as a FLOP experiment, not as a claim that the embedding width is the natural width of V_θ.

How MLP V_θ consumes the bank of context channels $\xi_t^{(k)}$

The original energy is defined as MLP (GELU stack) on the concatenated pair (ξ, h). Multi-ξ only widens the first linear map:

$V_\theta : \mathbb{R}^{(K+1)d} \to \mathbb{R}$, $\text{input} = \text{cat}\left(\xi_t^{(1)}, \ldots, \xi_t^{(K)},\right)$
Thus, `ScalarPotential` is $2d \to v_{\text{hidden}} \to \cdots \to 1$, while `ScalarPotentialMultiXi` is $(K+1)d \to v_{\text{hidden}} \to \cdots \to 1$. v_{hidden} (in code `v_hidden`) typically $3d - 4d$ e.g. With TinyStories we set $d = 256$, $v_{\text{hidden}} = 1024$. The α_k's are K scalars.

Because the MLP hidden layers mix all $(K+1)d$ coordinates, they can in principle use *cross-horizon correlations* (short-timescale ξ vs long-timescale ξ vs current h). That is why the MLP reference on TinyStories benefits from $K_\xi = 4$ relative to single- ξ whereas a diagonal-structured well bank often does not (in [1] : doubling K_ξ from 4 to 8 did not help SQ3; the MLP baseline at $K_\xi = 4$ reached lower PPL compared to its structured counterpart at $K_\xi = 8$).

Structured and Gaussian V_θ : flatten, then project

Structured potentials (quadratic (SQ3), isotropic and anisotropic Gaussians) were originally designed for single context vector of dimension ξ_d. The multi-ξ adapters do not use new geometry to incorporate the multiple channels in the well – they simply flatten the bank:

$\xi_{\text{flat}} = \text{reshape}(\xi) \in \mathbb{R}^{K_\xi d}$, $V = V_{\text{inner}}(\xi_{\text{flat}}, h)$

- `StructuredVThetaMultiXiAdapter` – quadratic / LSE mixtures ; inner `xi_d = K_xi*d` in code.
- `GaussianVThetaMultiXiAdapter` – same flatten into `mu_proj`, `a_proj`, and `w_proj`.

Well centers (`mu_proj`), precisions (`a_proj`), and mixture weights (`w_proj`) then become linear functions of the concatenated multi-resolution summary. A later variant, `MultiContextGaussianVTheta`, gives each $\xi^{(k)}$ its own well bank and sums $V = \sum_k V_k\left(\xi^{(k)}, h\right)$, so attractors are not forced to read a blurred $K_\xi d$ vector. That is an expressivity choice on top of K-EMA, not a change to the EMA itself. Specifically, the k-th well bank $V_k\left(\xi^{(k)}, h\right)$ in `MultiContextGaussianVTheta` is given as

$$V_k\left(\xi^{(k)}, h\right) = -\sum_{j=1}^{K_w} W_j^{(k)}\left(\xi^{(k)}, h\right) \qquad \text{(mcgvt.1)}$$

The j-th term $W_j^{(k)}\left(\xi^{(k)}, h\right)$ in the RHS of (mcgvt.1) represents a single Gaussian well:

$$W_j^{(k)}\left(\xi^{(k)}, h\right) = w_j^{(k)}\left(\xi^{(k)}\right) \exp\left(-\frac{1}{2}\left(h - \mu_j^{(k)}\right)^T P_j^{(k)}\left(h - \mu_j^{(k)}\right)\right) \quad \text{(mcgvt.2)}$$

Finally, the total structured potential V_θ , abbreviated with V, is the sum of all banks:

$$V = \sum_{k=1}^{K_\xi} V_k\left(\xi^{(k)}, h\right) \quad \text{(mcgvt.3)}$$

Note on structured V_θ : Well Banks, Well Centres, Well Precisions, Well Mixture weights and their projections

Definition *Bank*

Bank is one copy of structured well module: a mixture model (e.g. GMM) with its own projections, producing *one* scalar potential from *one* context vector ξ and hidden state h.

The number of wells in the bank is denoted with K_w in general. In older companion notes and in [1] in the context of using mixture models to construct V_θ we use K_{mix} to denote the number of wells in the mixture model.
For example, in `MixtureQuadraticVTheta` which represents the SQ3 variant of quadratic V_θ we use the following component mixture representation of the scalar potential $V_\theta = -\tau \log \sum_{k=1}^{K_{\text{mix}}} \pi_k e^{-E_k/\tau}$. In this formula we have K_{mix} components with weights π_k with energies E_k , combined by log-sum-exp.

How are banks assembled?

- **one bank (concat / joint)**: `V_theta.banks` has length 1. That bank's d_{in} is d or $K_\xi d$. The context channels are no longer per bank; all channels feed this one mixture.
- **additive multi-context:** `n_ctx` banks, `banks[m]` reads only $\xi^{(m)}$ with $d_m = d$. Then $V_\theta = \sum_m V_{\text{bank}}^{(m)}$. For example, in many of our recent models we use 5 banks with 8 wells each abbrev. 5 heads × 8 wells.

One Bank (Concat / Joint)
Implemented via *Concat Adapters* in quadratic V_θ (SQ3) and early Gaussian implementations.
Flatten $\xi \mapsto \mathbb{R}^{K_\xi d}$, one mixture, $d_{\text{in}} = K_\xi d$. No `V_theta.banks` or not used in the usual way.
Concat adapters are :
- `StructuredVThetaMultiXiAdapter` – used in the quadratic V_θ implementation SQ3 `MixtureQuadraticVTheta`.
Used on Multi- ξ SPLM / PARFLM / Fock-PARFLM v2.1 (A1–A5, Phase 4 OWT SQ3)
- `GaussianVThetaMultiXiAdapter` - used in isotropic `MixtureGaussianVTheta` and in `SARFGaussianVTheta`. Used on hybrid Gaussian–quadratic notebook and in Phase 5 SARF/Gaussian concat arms.
There is also a newer implementation which uses one *Joint Bank* (see [9])
- `JointContextAnisotropicGaussianVTheta` – used in `AnisotropicDepthConditionedGaussianVTheta` , code : model_aniso_gaussian_vtheta.py.

The well centres are denoted with $\mu_k \in \mathbb{R}^d$. The diagonal precisions are denoted with $a_k \in \mathbb{R}^d_{>0}$ and represent the inverse variances per coordinate. The mixture weights are denoted with $w_k \in \mathbb{R}$.

More recently we have studied GMMs such as `MixtureGaussianVTheta` and `AnisotropicMixtureGaussianVTheta`. So, the inputs, parameters and outputs of those mixture models are:

- inputs : $\xi \in \mathbb{R}^{d_{\text{in}}}$, $h \in \mathbb{R}^d$
- parameters : the bank's `mu_proj`, `a_proj`, `w_proj`, (and `B_proj` if anisotropic)
- output: K_w wells $\{(\mu_k, P_k, w_k)\}_{k=1}^{K_w}$ and $V_{\text{bank}}(\xi, h) = -\sum_{k=1}^{K_w} w_k \exp\left(-\frac{1}{2}(h-\mu_k)^T P_k (h-\mu_k)\right)$

There are three maps, also denoted as *projections*, which are ordinary `nn.Linear` layers and their goal is to turn context only into Gaussian-mixture parameters:

In the method `MixtureGaussianVTheta::_components` the following happens

$\mu = \text{reshape}\big(\text{mu_proj}(\xi)\big) \in \mathbb{R}^{K_w \times d}$
$a = \text{softplus}\big(\text{a_proj}(\xi)\big) + 10^{-4} \in \mathbb{R}^{K_w \times d}$ (then optional clamp)

$w = w_{\text{scale}} \cdot \text{softmax}\big(\text{w_proj}(\xi)\big) \in \mathbb{R}^{K_w}$

The potential for a single bank for isotropic Gaussian family is then:

$$V(\xi, h) = -\sum_{k=1}^{K_w} w_k(\xi) \exp\left(-\frac{1}{2}\sum_{i=1}^{d} a_{k,i}(\xi)\left(h_i - \mu_{k,i}(\xi)\right)^2\right) \quad \text{(igp.1)}$$

So,

`mu_proj` tells us where is the well k. It is *affine* in ξ because `mu_proj` is implemented as `nn.Linear`, that is: $\mu(\xi) = W_\mu \xi + b_\mu$.
`a_proj` tells us how sharp well k is along each axis. softplus keeps a > 0. With `init_log_precision` $= \log d$, $a \approx \frac{1}{d}$ so $\sigma_{\text{eff}} \approx \sqrt{d}$, matching `LayerNorm` scale. `a_proj`, being `nn.Linear`, renders $a(\xi) = \text{softplus}(W_a \xi + b_a) + 10^{-4}$ which clearly is not affine transformation.
`w_proj` tells us which wells of the bank are ***on*** for this context ; softmax scales $\sum_k w_k = w_{\text{scale}}$ which is usually 1.

Anisotropic banks add an extra map : `B_proj`: $d_{\text{in}} \to K_w \cdot d \cdot r$, reshaped to $B_k \in \mathbb{R}^{d \times r}$ so the precision is $\text{diag}(a_k) + B_k B_k^T$. Here r is the rank of the off-diagonal precision factor for each well – it is given with constructor argument rank , typically is 4 on OWT anisotropic Gaussian Fock models. Recall, that each anisotropic Gaussian well k has a factor $B_k \in \mathbb{R}^{d \times r}$ obtained from `B_proj`. The precision (inverse covariance) P_k is the diagonal plus a rank-r PSD update:

$$P_k = \text{diag}(a_k) + B_k B_k^T \in \mathbb{R}^{d \times d}$$

$B_k B_k^T$ has rank of at most r, so the well is an ellipsoid that can tilt off the coordinate axes (correlations between hidden dimensions). `rank` = 0 drops `B_proj` and $P_k = \text{diag}(a_k)$ and we recover the isotropic bank.

Contrast with MLP V_θ
The MLP first layer sees `cat`(ξ, h). Gaussian projections see ξ only. h appears only in $(h - \mu)^T P (h - \mu)$. That is why centres can be read off as $\mu(\xi)$.

Where Does Flattening Happens?

The inner bank always assumes one context vector of length d_{in}. The multi-ξ context state is a tensor of shape (B, T, K_ξ, d),

$$\xi \in \mathrm{R}^{B \times T \times K_\xi \times d}.$$

The concat adapters `StructuredVThetaMultiXiAdapter` and `GaussianVThetaMultiXiAdapter` do not invent a new geometry for this tensor. They reshape the last two axes,

$$\xi_{\text{flat}} = \text{reshape}(\xi) \in \mathrm{R}^{K_\xi d},$$

and call the inner bank as $V = V_{\text{inner}}(\xi_{\text{flat}}, h)$. The hidden state h remains a separate argument. It is **not** concatenated into the well-parameter maps. Those maps see only the flattened context, with input width `xi_d` $= K_\xi d$:
`mu_proj` maps $K_\xi d$ to $K_w d$, `a_proj` maps $K_\xi d$ to $K_w d$, and `w_proj` maps $K_\xi d$ to K_w.
Every well parameter is therefore a dense linear mixture of **all** horizons. The model cannot dedicate an attractor to a single timescale: there is no well that "belongs" to the hundred-token exponential moving average, because every centre, precision, and mixture weight is allowed to depend on every channel. With the usual recipe $K_\xi = 4$ and

$K_w = 8$ this produces eight attractors on a blurred $4d$-dimensional summary, not thirty-two or forty horizon-specific centres.

A tiny example makes the wiring concrete. Take $K_\xi = 2$ channels, $K_w = 2$ wells, and $d = 3$. The concat adapter stacks $\xi^{(1)}, \xi^{(2)} \in \mathrm{R}^3$ into $\xi_{\text{flat}} \in \mathrm{R}^6$. Then `mu_proj` is an affine map $\mathrm{R}^6 \to \mathrm{R}^6$ whose six outputs are reshaped into two centres in R^3. Each coordinate of μ_1 can depend on both the short-horizon and the long-horizon context. The additive alternative, developed earlier in this section and implemented by `MultiContextGaussianVTheta`, would instead give each channel its own bank: bank 1 is an affine map $\mathrm{R}^3 \to \mathrm{R}^6$ that sees only $\xi^{(1)}$, bank 2 is a twin map that sees only $\xi^{(2)}$, and the two banks together produce four centres, two per horizon.

Three things are **not** flattened and confusing them with the adapter is a common source of misreading.

• The particle coordinate h never enters `mu_proj`, `a_proj`, or `w_proj`. Those three maps are functions of context only. The hidden state appears later, in the quadratic form or Gaussian bump that is evaluated at $h - \mu_k(\xi)$. That is why centres can be read off by calling `attractor_centres` without differentiating through h.

• The well index k is not a separate network. A single wide linear map writes a vector of length $K_w d$ (or K_w for the weights), and a reshape peels that vector into K_w blocks. Multi-context wiring duplicates the whole bank per channel; it does not allocate one MLP per well.

• The weight head `w_proj` emits K_w logits, not $K_w d$ numbers. Mixture weights are per well, not per hidden coordinate.

Flattening, in this architecture, is only an adapter that sets the input width of those three heads to $K_\xi d$ instead of d. The later OpenWebText Gaussian runs that quote "five heads times eight wells" use the other wiring: no flattening, five separate triples of (`mu_proj`, `a_proj`, `w_proj`), and a sum of five bank potentials. That design is parameter-neutral with the concat baseline **when both keep the same K_w per bank**: five maps $d \to 8d$ have the same number of weights as one map $5d \to 8d$, but they produce forty centres instead of eight, and they are block-diagonal across horizons rather than dense in the concatenated context. A dense map $5d \to 40d$ that tried to match the forty centres in a **single** concat bank would be five times heavier; that is not the concat baseline actually used.

Does the anisotropic Gaussian benefit beyond 4 Channels?

There is some benefit to giving an anisotropic Gaussian more than four K-EMA channels, but the returns diminish sharply and can become negative. They diminish faster than they do for the MLP V_θ. The reason is not that four is a magic number for language. It is that the additive multi-context Gaussian and the MLP spend extra channels in different ways.

In the additive design the force is a sum of per-bank forces,

$$f = -\nabla_h V_\theta = \sum_{m=1}^{n_{\text{ctx}}} f^{(m)}(\xi^{(m)}, h).$$

Each bank is expressive **within** its own horizon: centres $\mu_k^{(m)}(\xi^{(m)})$ and precisions $P_k^{(m)}(\xi^{(m)})$ are nonlinear functions of that channel. What the sum cannot represent is a conjunction across horizons. There is no attractor that fires only when the two-token channel says one thing **and** the two-hundred-token channel says another. Adding a channel therefore helps only if that horizon carries a useful **standalone** signal. It does not buy interaction terms. The MLP, by contrast, concatenates every $\xi^{(m)}$ with h and sends the joint vector through a shared nonlinearity, so gating, disagreement detectors, and other cross-horizon features are available in principle.

Adjacent exponential moving averages are also strongly correlated. Channels with decays such as $\alpha = 0.99$ and $\alpha = 0.995$ are nested low-pass filters; every kernel passes the constant (direct-current) mode, so the effective number of independent directions in the bank is smaller than the nominal K_ξ. An extra Gaussian bank that sees a near-duplicate of an existing $\xi^{(m)}$ still adds a full set of well parameters and a full contribution to the force. The additive structure cannot rotate that redundancy away. The MLP's first layer can down-weight or linearly combine correlated inputs at cost $d \cdot d_V$ per extra channel, without cloning a curvature machine.

That last point is specific to the anisotropic family. Every anisotropic bank has its own `B_proj`, producing a rank-r update $B_k B_k^\top$ in the precision. Extra channels increase the independent off-diagonal curvature that the integrator

has to confine (the “mechanism-A stiffness” of the CfC-BAOAB notes). The MLP has no per-channel ellipsoid at all. On OpenWebText Fock-PARFLM runs the K-EMA bank is therefore almost always left at four channels. For the MLP that choice makes the first layer a map $5d \to d_V$. For the additive Gaussian it is a compromise between covering a few genuinely different timescales and not cloning correlated force fields.
That is why proposals such as HiPPO-LegT or S4D, discussed in the multi-channel ξ notes, try to replace raw EMAs by more orthogonal filters rather than by simply taking K_ξ larger. Orthogonalising the bank is a change to how ξ is computed. It is not a substitute for giving V_θ a joint, rather than additive, reading of those channels — the subject of the next subsection and of [9].

Can we do something better than flattening?

Yes. Two alternatives are already in the code, and they attack different parts of the concat-adapter limitation.
The first is the additive multi-context wiring already described: `MultiContextGaussianVTheta` and its depth-conditioned and anisotropic relatives. Flattening is abandoned; each channel owns a bank of K_w wells; the potential is the sum of the bank scalars. That restores horizon-specific attractors at the same parameter count as a concat bank with the same K_w. It does **not** restore cross-horizon conjunctions, which is why extra channels still saturate, as the previous subsection argued.
The second is a **joint** bank whose well parameters are functions of the concatenated context, implemented as `JointContextAnisotropicGaussianVTheta` and selected on `AnisotropicDepthConditionedGaussianVTheta` by `coupling='joint'`. Analyticity of the force requires only that μ_k, a_k, B_k and w_k depend on ξ and not on h. They are allowed to be arbitrary joint functions of all channels. A single projection $n_{\text{ctx}}d \to K_w d$ is the simplest such function: every well can see every horizon, so additive separability is broken, while the force, the Hessian in h, and the CfC-BAOAB harmonic split remain closed form. To match the forty attractors of a 5×8 additive stack one sets $K_w = 40$ on that one bank, accepting a denser map.
Companion note [9] (`Analytic_Multi_Channel_Integration_in_Structured_Vtheta.md`) develops the rest of the ladder — conjunctive gates, product-of-experts fusion of Gaussian factors, bilinear cross-features — and the curvature caveat: fusing channels concentrates stiffness rather than cloning it. The joint projection is the drop-in prototype; the other rungs remain design options. Nothing in that note changes the K-EMA computation of ξ itself. Geodesic structure still lives in how the integrator walks $V_\theta(h;\xi)$, not in how the prefix is averaged.

Depth conditioning and context banks in structured V theta

The structured Gaussian used on OpenWebText Fock-PARFLM is not a concat adapter. It is a **depth-conditioned context-bank** module: several well banks, one per K-EMA channel, sharing their parameters across Verlet layers except for a small, learned shift of the context. The isotropic form is `DepthConditionedMultiContextGaussianVTheta` in `model_gaussian_vtheta.py`. The anisotropic form is `AnisotropicDepthConditionedGaussianVTheta` in `model_aniso_gaussian_vtheta.py`. Both read the same multi-channel tensor $\xi \in \mathrm{R}^{B\times T\times n_{\text{ctx}}\times d}$ and the same particle coordinate $h \in \mathrm{R}^d$.

Context banks

Write n_{ctx} for the number of banks and K_w for the number of wells inside each bank. The OpenWebText default is $n_{\text{ctx}} = 5$ and $K_w = 8$, hence forty Gaussian terms. Bank m sees only its own channel $\xi^{(m)} \in \mathrm{R}^d$. Its three maps are therefore narrow,
Bank m therefore has `mu_proj` mapping d to $K_w d$, `a_proj` mapping d to $K_w d$, and `w_proj` mapping d to K_w, and the total potential is the sum already written as (mcgvt.3),

$$V_\theta(\xi, h) = \sum_{m=1}^{n_{\text{ctx}}} V^{(m)}(\xi^{(m)}, h).$$

Each centre is a function of a single timescale. The linear mass of the centre maps is the same as that of a concat adapter with the same K_w, but the forty centres are not forty linear mixtures of a blurred $5d$ vector; they are eight centres on channel 1, eight on channel 2, and so on. Depth conditioning, below, does not add a fourth projection. It only shifts the input that these three maps already see.

Depth codes as a per-layer shift

Fully untying V_θ across the L integration layers would copy the entire bank stack L times. At $d = 384$, $K_w = 8$, $n_{\text{ctx}} = 5$ and $L = 16$ that is tens of millions of extra parameters, almost all of them in the dense $d \to K_w d$ maps. The depth-conditioned module keeps **one** shared stack and introduces a learned table

$$e \in \mathrm{R}^{L \times n_{\text{ctx}} \times d},$$

the `depth_code` parameter, initialised from $\mathcal{N}(0, 0.02^2)$. Before the bank projections at layer ℓ one replaces

$$\xi^{(m)} \; arrow \; \xi^{(m)} + e_{\ell,m}.$$

The code is constant with respect to h, so the force remains $f = -\nabla_h V_\theta$ of a scalar; only the conditioning has changed. The extra cost is $Ln_{\text{ctx}}d$ parameters (about thirty thousand at the numbers above), not a second copy of the banks. The well **shapes** stay tied across depth; what varies with ℓ is which view of ξ those shapes are evaluated at. The layer index is not an extra argument of `forward`. The model's per-layer step writes it into `_active_layer` (`install_depth_routing` on `FockMultiXiPARFLM`), including during gradient-checkpoint recomputation, and the module reads that pointer when it shifts ξ.

Where the forty wells sit before training

The forty wells are not a stored lookup table of points. Each centre is an affine image of its channel,

$$\mu_{m,k}(\xi^{(m)}) = \mathrm{reshape}(W_m \xi^{(m)} + b_m)_k \in \mathrm{R}^d.$$

There is no buffer of frozen PMI anchors — that construction belongs to `SARFGaussianVTheta`, a different class. The biases b_m are initialised to zero in both the isotropic and the anisotropic families. Consequently, if the context vanished, every centre would sit at the origin: a forty-fold degeneracy. On a real first batch, ξ is a K-EMA of embeddings drawn from $\mathcal{N}(0, 0.02^2)$ plus positional codes, so $\mu = W\xi$ is only a tiny random scatter about zero, not a lattice, a simplex, or a set of token prototypes.

The two families differ in how small that scatter is. Isotropic `mu_proj` uses Gaussian weights of standard deviation 0.02. Anisotropic `mu_proj` uses Xavier uniform on `Linear(d, K_w d)`, whose bound is $\sqrt{6/(d + K_w d)} = \sqrt{2/(3d)}$ when $K_w = 8$, about 0.042 at $d = 384$. In both cases ξ itself is $O(1/\sqrt{d})$ after LayerNorm-scale embeddings, so all forty centres start on top of each other near the origin. They do not tile the LayerNorm sphere.

The mixture weights start uniform. On the anisotropic bank that is exact: `w_proj` is initialised with zero weight and zero bias, so the softmax logits are identically zero and each well in a bank carries mass $1/K_w = 1/8$. Precisions are initialised from `init_log_precision` $= -\log d$, which gives an effective width $\sigma_{\text{eff}} \approx \sqrt{d}$. On a LayerNorm sphere of radius $\sqrt{d}$ that is a stack of **wide, overlapping bowls** around the origin, not forty separated basins.

Saying that "forty attractors exist from step 0" is a statement about the **parameterisation** — there are forty Gaussian terms in the formula — not about geometry. Distinct centres have to be learned as the maps W_m and the depth codes move.

They are in particular **not** initialised to random points on the LayerNorm sphere, to vocabulary or embedding prototypes, to a k-means of hidden states, or to SARF PMI-peak anchors. A direct check is to call `V_theta.attractor_centres(xis)` on the first training batch, before `optimizer.step()`. One should see forty vectors with Euclidean norms much smaller than 1 and pairwise distances much smaller than $\sqrt{d}$, all clustered at the origin.

Analytic Multi-Channel Integration: a Ladder of Couplings

The additive multi-context design just described gives each channel its own well bank and sums the per-channel potentials:

$$V^{\text{add}}(h;\xi) = \sum_{m=1}^{n_c} V^{(m)}(\xi^{(m)}, h), \qquad V^{(m)}(\xi^{(m)}, h) = -\sum_k w_k^{(m)} \exp\!\big(-1/2\,(h-\mu_k^{(m)})^\top P_k^{(m)} (h-\mu_k^{(m)})\big).$$

Because the channel sum sits outside the exponent, this is a mixture: the potential is attracted to a match on channel 1 *or* channel 2 *or* … Its Hessian in the concatenated channel input $\xi = [\xi^{(1)}; \ldots; \xi^{(n_c)}]$ is block-diagonal — every cross-horizon second derivative $\partial^2 V / \partial \xi^{(i)} \partial \xi^{(j)}$ with $i \neq j$ is exactly zero. No amount of extra channels can build a feature that fires only when several horizons agree.

The analyticity hinge

Everything that makes the structured V_θ tractable — a closed-form force $-\nabla_h V$, a closed-form Hessian, and the CfC-BAOAB harmonic split of the section above — follows from one fact: the well parameters are projections of ξ *only*, never of h. That is the entire content of _components(self, xi) in every bank implementation described so far: mu, a, w, and B are each computed from xi alone. So the potential is exactly Gaussian in h regardless of how those parameters were computed from ξ — they may be arbitrary joint functions of all channels: a shared projection, a bilinear form, an MLP, a product-of-experts fusion. The additive design simply chose the most restrictive such function, one channel per bank. What follows is a ladder of less restrictive choices, each of which keeps the well parameters h-independent and therefore keeps the analytic force and the CfC-BAOAB split.

(A) Joint projection — a single bank over the concatenated channels. Replace n_c per-channel banks with one bank whose projections read $\xi = [\xi^{(1)}; \ldots; \xi^{(n_c)}] \in \mathbb{R}^{n_c d}$:

$$(\mu_k, a_k, B_k, w_k) = \text{proj}(\xi^{(1)}, \ldots, \xi^{(n_c)}).$$

Every well's centre, precision, and weight now depends on all horizons jointly; the additive design is a strict special case, so expressivity only rises. This is the option implemented in production as JointContextAnisotropicGaussianVTheta, discussed below.

(B) Conjunctive gating. Make the (h-independent) weight a joint gate, $w_k = \prod_m \sigma\left(s_k^{(m)}(\xi^{(m)})\right)$. Since w_k multiplies the whole bump, a well fires only when its centre matches h *and* the gate is on — an AND across horizons.

(C) Product-of-experts fusion. Move the channel sum *inside* the exponent by multiplying the per-channel Gaussian factors. A product of Gaussians is a Gaussian, so this collapses to a single well analytically, developed next.

(D) Bilinear coupling. Let μ_k or a_k depend on cross-features such as $\xi^{(i)} \odot \xi^{(j)}$ (Hadamard) or a low-rank bilinear map — the cheapest way to inject a specific conjunction into where a well sits.

(E) Analytic marginalisation. If "integrate" is meant literally, put a channel-informed Gaussian prior on the centre and integrate it out; a convolution of Gaussians stays Gaussian:

$$\int \mathcal{N}(h;\mu,P^{-1})\,\mathcal{N}(\mu;\bar{m}(\xi),\Lambda(\xi))\,\mathrm{d}\mu = \mathcal{N}(h;\bar{m},P^{-1}+\Lambda).$$

Product-of-experts fusion in closed form

Option (C) is worth deriving explicitly, since it shows quantitatively what "conjunction" means. For a fused well k, let each channel m propose a Gaussian factor with centre $\mu_k^{(m)}$ and precision $P_k^{(m)}$. Multiplying the factors adds their exponents:

$$E_k(h) = \sum_{m=1}^{n_c} (h-\mu_k^{(m)})^\top P_k^{(m)} (h-\mu_k^{(m)}).$$

Completing the square collapses this to a single anisotropic Gaussian. With

$$P_k = \sum_m P_k^{(m)}, \qquad \mu_k^\star = P_k^{-1} \sum_m P_k^{(m)} \mu_k^{(m)}, \qquad c_k = \sum_m \mu_k^{(m)\top} P_k^{(m)} \mu_k^{(m)} - \mu_k^{\star\top} P_k \mu_k^\star,$$

the fused energy and potential are

$$E_k(h) = (h - \mu_k^\star)^\top P_k (h - \mu_k^\star) + c_k, \qquad V^{\text{fuse}}(h;\xi) = -\sum_k w_k^\star \exp(-1/2\,(h - \mu_k^\star)^\top P_k (h - \mu_k^\star)),$$

with $w_k^\star = w_k \exp(-1/2\, c_k)$. Fusing channels by product therefore yields one well whose centre is the precision-weighted (Bayesian) fusion of the channel centres and whose precision is the sum of the channel precisions. The constant c_k, which grows when the channels disagree, folds into the weight — so a fused well is strong only where the horizons agree, which is exactly the semantics of conjunction.

Cost note. Exact fusion (C) needs P_k^{-1} per well per token — tractable via Woodbury since $P_k = \text{diag} + \text{low-rank}$, but not free — and the fused low-rank factor has rank $n_c r$. Option (A) reaches the same expressive class, a single anisotropic Gaussian whose parameters depend jointly on all channels, without the inverse and with a lighter rank-r factor, simply by *learning* (μ_k, a_k, B_k) from the concatenated context. That is why the production prototype implements (A): (C) is the principled explanation, (A) is the cheap realisation.

The catch: fusion concentrates curvature

Genuine conjunction has a price, and it lands squarely on the stiffness question the anisotropic implementation already raised. In (C), $P_k = \sum_m P_k^{(m)}$: the fused well is stiffer than any single channel, and its low-rank part accumulates to rank $n_c r$. The dial that controls this:

Fusion	Precision	Semantics	Stiffness
product (strict AND)	sum of channel precisions	firm conjunction	grows with channel count
convex (weighted)	convex combination, weights summing to 1	soft interpolation	bounded by the stiffest channel
joint projection (A)	learned directly	learned mixture of AND / OR	controllable via precision_lr_max

Whichever coupling is chosen, pairing it with the existing precision_lr_max cap is not optional: coupling makes the model more expressive *and* more spike-prone, and the cap is what turns that added expressivity into a stable equilibrium rather than a blow-up.

The harmonic split stays valid

The CfC-BAOAB harmonic split consumes only the final per-well numbers P_k, μ_k, g_k; it does not care how they were produced. Joint coupling is therefore fully compatible with baoab_cfc and baoab_cfc_lowrank. The diagonal spring and PSD low-rank operator remain

$$k_{\text{diag}} = \sum_k g_k\ \text{diag}(P_k), \qquad \mathcal{L} = \sum_k g_k\, B_k B_k^\top = G G^\top, \qquad g_k = w_k \exp(-1/2\,(h - \mu_k)^\top P_k (h - \mu_k)),$$

with G the concatenation of the per-well factors. For a joint bank with K wells the low-rank footprint is Kr modes — the same or fewer than the additive variant's $n_c Kr$ at equal K.

The prototype: JointContextAnisotropicGaussianVTheta

Option (A) is implemented as a drop-in replacement for the additive multi-context module, with an identical constructor signature and identical public methods. Internally it is a single mixture bank with input width $\mathrm{xi_d} = n_c \cdot d$, so every analytic method — the force, the harmonic split, context_components, attractor_centres — is inherited by delegation rather than reimplemented. The depth-conditioned wrapper exposes the choice as a coupling flag on construction, defaulting to the additive behaviour already described; setting coupling="joint" is a one-line switch to the joint bank. Because the joint bank has K joint attractors versus the additive design's $n_c K$, matching the additive attractor budget means setting $K = n_c \cdot K_{\text{additive}}$.
A test suite verifies the properties that matter for trusting this as a drop-in: the analytic force matches autograd to 10^{-4}; the diagonal/low-rank harmonic split reproduces the true force exactly, with the low-rank operator positive semi-definite; rank 0 reduces to the exact full force; and — the load-bearing check — the joint bank's cross-channel Hessian block is measurably nonzero while the additive bank's is exactly zero, confirming in code that coupling is actually happening rather than merely being architecturally possible.

What this predicts, and how to test it

A clean comparison on a fixed corpus, holding channel count, V_ϕ, schedule, and seed constant while toggling only the coupling, makes four falsifiable predictions: joint coupling should improve perplexity at fixed channel count and equal parameter budget; joint coupling should keep improving with more channels in the regime where the additive design plateaus (the diminishing-returns argument of the section above); joint coupling paired with precision_lr_max should train without watchdog reloads; and the offline channel-Hessian coupling gap should track the perplexity gap across seeds, distinguishing coupling itself from a mere capacity increase. The predicted failure mode, if confinement is skipped, is exactly the curvature concentration derived above showing up as additional gradient spikes — the expected and testable cost of the added expressivity, not a surprise.

References

[1] Semantic Simulation: A Prescriptive Lagrangian Framework for Efficient Semantic Inference, D.Gueorguiev, 2026
[2] Semantic Attractor Extraction from V_θ, companion note to [1]
[3] Structured V_θ: Theory, Derivation, and Analysis, companion note to [1]
[4] Potential Wells, Temperature, and the Boltzmann-Gibbs Connection, companion note to [1]
[5] Training Instabilities in Fock-PARFLM v2.1 with Structured V_θ, companion note to [1]
[6] Fock-PARFLM vs GPT-2 on OpenWebText: Predicted Results and Next Steps, companion note to [1]
[7] ξ-Bottleneck Diagnosis: Multi-ξ Fock-PARFLM v2.1 on OpenWebText, companion note to [1]
[8] Deep Dive in the SPLM model and code, companion note to [1]
[9] Analytic Multi-Channel Integration in a Single Structured V_theta Well, companion note to [1]
[10] Curvature Diagnostics and Rank Selection for Anisotropic Gaussian V_theta, companion note to [1]

Appendix

Note on logit ℓ and embedding matrix E in SPLM

A logit ℓ is the raw, unnormalized score that the model assigns to each token before converting it to probability.

Concretely, for a hidden state $h^* \in \mathrm{R}^d$ of the final layer (denoted by superscript * or alternatively by subscript L as h_L) and embedding matrix $E \in \mathrm{R}^{V \times d}$, vectors in R^d are columns. The logit for token v is the inner product

$$l_v = e_v^\top h^* \in \mathrm{R}(\log.1)$$

where $e_v \in \mathrm{R}^d$ is the embedding vector of token v, i.e. row v of E written as a column: $e_v \equiv (E_{v,*})^\top$. This is the Euclidean inner product of two length-d vectors — a single scalar measuring how aligned h^* is with that token's direction. The same map in matrix form is $Eh^* \in \mathrm{R}^V$ (see (log.5) and the tied-head identity $z = Eh_L$ below). The dotted operators · used later for logits are this inner product when both factors are length-d vectors; they are **not** Hadamard products.

We can make the token position explicit by subscripting h^* explicitly with the sequence position t:

$$l_{t,v} = e_v^\top h_t^* \in \mathrm{R}(\log.2)$$

So both ℓ_v and $\ell_{t,v}$ denote the same scalar score (the logit) for vocabulary token v after position t in the sequence. Both h^* and h_t^* denote the hidden state of the *final layer* at token position t in the sequence.
We will use both notations to denote the same quantity depending on the context where the token position can be omitted or should be made explicit.

The full logit matrix for one forward pass is then all these inner products stacked:

$$l_{t,v} = e_v^\top h_t^* \in \mathrm{R} \text{for all } t \in \{1, \dots, T\}, v \in \{0, \dots, V-1\}(\log.3)$$

or equivalently as the matrix product, with $H^* \in \mathrm{R}^{T \times d}$ whose t-th row is $(h_t^*)^\top$

$$L = H^* E^\top \in \mathrm{R}^{T \times V}(\log.4)$$

We get one such number for every token in the vocabulary, so the full logit vector at a single position (column convention, matching $z = Eh_L$ below) is

$$l = Eh^* \in \mathrm{R}^V(\log.5)$$

These raw scores have no direct probabilistic meaning yet – they can be any real number, positive or negative, and they do not sum to 1. The word "logit" comes from statistics where it refers to the log-odds of probability, but in deep learning it has been adopted more loosely to mean any pre-softmax score. To turn logits into proper probability distribution we apply `softmax`:

$$P(v) = \frac{\exp \ell_v}{\sum_{v'} \exp \ell_{v'}} \quad \text{(lpr.1)}$$

In this context the term *top token* is the token our model considers most likely which is $\underset{v}{\operatorname{argmax}}\, \ell_v$.

In the attractor extraction context, the expression *read out through the **tied** LM head* means computing these logits for the converged h^* and then looking at which tokens have the highest scores – those are the tokens the attractor represents semantically.

Note on SARF-faithful mass

The term "SARF-faithful mass" denotes two separate implementations of semantic mass which are related to the Semantic Simulation framework descriptive part.

Component 1: SARF-faithful ξ (Section 15.16 of [1])

In the baseline SPLM, the context vector ξ_t (causal cumulative mean of the embedding sequence) is computed once from the input embeddings before the integration loop and held fixed for all L Euler steps. The SARF-faithful variant re-derives ξ from the *current* hidden states at every integration step l:

$$\xi_t^{(l)} = \mathrm{CausalMean}(h_1^{(l)}, \ldots, h_t^{(l)}) = \frac{1}{t}\sum_{s=1}^{t} h_s^{(l)} \text{ (avg.1)}$$

$\mathrm{CausalMean}$ guarantees that each token at position t sees only the mean of itself and all preceding tokens – never future tokens. This literally realizes the time-dependent reinforcement field with force $\vec{f}(h,t)$ from Section 6 of [1]. In this context the reinforcement field force is simply

$$\vec{f}(h,l) = -\nabla_h V_\theta(\xi, h) \quad \text{(rif.1)}$$

In the multi-channel ξ variant the simple mean is replaced by exponential moving averages with different decay rates α_k:

$$\xi_{t,k}^{(l)} = \alpha_k \cdot \xi_{t-1,k}^{(l)} + (1-\alpha_k) \cdot h_t^{(l)} \qquad \text{(ema.1)}$$

Each channel k has its own decay α_k (from the logs of a currently running experiment $[0.027, 0.420, 0.588, 0.857, 0.974]$), giving different memory horizons:

Channel	α_k	Effective Window	What it captures
0	0.027	~ 1 token	Nearly instantaneous (current token)
1	0.420	~ 2 tokens	Very recent context
2	0.588	~ 2-3 tokens	Short-range context
3	0.857	~ 7 tokens	Medium-range context
4	0.974	~ 38 tokens	Long-range context

The effective window is approximately $1/(1-\alpha_k)$. The full context vector $\xi_t^{(l)}$ is the concatenation of all channels (a length-Kd vector; equivalently a (K, d) stack before the adapter flattens it):

$$\xi_t^{(l)} = (\xi_{t,1}^{(l)}; \xi_{t,2}^{(l)}; \ldots; \xi_{t,K}^{(l)}) \in \mathrm{R}^{Kd}$$

Component 2: logfreq per-token mass (as in Section 15.17 of [1])

The Euler-Lagrange integrator has inertial mass term $\mathfrak{m}$ that controls how strongly the forces accelerate the hidden state.

"SARF-mass" is the combination of two orthogonal ablations stacked on top of the basic SPLM — both documented in the paper (§14.13 and §14.14) and the best standalone-SPLM configuration in the paper.

Component 1 — SARF-faithful ξ (§14.13)

In the baseline SPLM, the context vector ξ_t (causal cumulative mean of the embedding sequence) is computed once from the input embeddings before the integration loop and held fixed for all L Euler steps. The SARF-faithful variant re-derives ξ from the current hidden states at every integration step ℓ:

$$\xi^{(\ell)} = \mathrm{CausalMean}(h_1^{(\ell)}, \ldots, h_t^{(\ell)})$$

This literally realises the time-dependent reinforcement field from §6 of the theory (layer ℓ plays the role of time). The only code change is one extra line inside the integration loop — the force is still $-\nabla_h V_\theta(\xi, h)$, nothing else changes. On Tiny Shakespeare, this single change gives −33% val PPL at identical parameter count.

Component 2 — logfreq per-token mass (§14.14)

The Euler–Lagrange integrator has a kinetic prefactor m (inertial mass):

$$v_i = \frac{v_{i-1} + \Delta t \cdot f/m}{1 + \Delta t \cdot \gamma}$$

The baseline SPLM uses a single global scalar m. The logfreq variant makes this per-token and information-theoretically grounded:

$$m_t = \mathrm{softplus}(b_m + \alpha \cdot (-\log \hat{p}(x_t))) + \epsilon$$

$\hat{p}(x_t)$ is the add-one-smoothed unigram frequency on the training corpus (precomputed once, frozen). The only new learnable parameter is the scalar scale α. This makes rare / surprising tokens heavier — they resist being pushed away from their initial position by the force field, which is exactly what the Lagrangian framework prescribes for semantically loaded tokens.

The "SARF" in "SARF-mass" signals that this mass variant was tested on top of the SARF-faithful ξ. An alternative Variant A (learned linear projection from the token embedding to a scalar mass) was also tested but underperforms logfreq by ~27% on Tiny Shakespeare, because at this data scale the frozen informational shape wins over a learned shape that has to discover both shape and scale from scratch.

Combined result (§4.4 of Training_and_Inference_with_SPLM.md)

variant	val ppl	Δ vs baseline
Fixed-ξ SPLM	287.4	—
SARF-faithful only	192.2	−33%
SARF + embed-head mass (A)	222.9	−22%
SARF + logfreq mass (B)	160.6	−44%

This is the §14 flagship: one extra scalar parameter α plus a frozen vocabulary-sized surprisal tensor, reducing the gap to the matched GPT-2 baseline from ~2.0× to ~1.13×.

Note on learning in Fock-PARFLM using structured V_θ energy landscape

Fock-PARFLM set up in an energy landscape composed by structured V_θ wells learns in a variety of ways-

For example let us consider Gaussian wells with learned anchors. The code defining the mixture of Gaussian wells with learned anchors is in the class `MixtureGaussianVTheta`. The optimized anchors are learned linear function of the context ξ:

model_gaussian_vtheta.py, lines 93-95:

```
self.mu_proj = nn.Linear(in_d, K * d)
self.a_proj = nn.Linear(in_d, K * d)
self.w_proj = nn.Linear(in_d, K)
```

So $\mu_k(\xi) =$ `self.mu_proj(xi)`, $a_k(\xi) =$ `F.softplus(self.a_proj(xi))`, $w_k(\xi) =$ `F.softmax(self.w_proj(xi))` The centres move with every token context. Note that in class `SARFGaussianVTheta` , corresponding to fixed Gaussian well anchors, we have an alternative implementation where the anchors are frozen buffers and only `log_sigma + w_proj` are learned. So in this scenario of fixed anchors V_θ's learnable parameters are the weights and biases of those three projections – not the well positions directly.

While the model learns through a semantic energy landscape, the Gaussian wells (V_θ) represent only a small fraction of the total learnable parameters. In the specific run mentioned, the model has approximately 33.9 million parameters in total, dominated by tied token embeddings, the pairwise term (V_ϕ), and the Fock register machinery.

Clarification on "optimised anchors"

In the `MixtureGaussianVTheta` variant the centres are a learned linear function of the context ξ. Implementation (`model_gaussian_vtheta.py`, lines 93–95):

```
    self.mu_proj = nn.Linear(in_d, K * d)
    self.a_proj  = nn.Linear(in_d, K * d)
    self.w_proj  = nn.Linear(in_d, K)
```

$$\mu_k(\xi) = \text{mu_proj}(\xi)$$

$$a_k(\xi) = \text{softplus}(\text{a_proj}(\xi))$$

$$w_k(\xi) = \text{softmax}(\text{w_proj}(\xi))$$

Because these parameters are functions of ξ, the centres move with every token's context. By contrast, `SARFGaussianVTheta` uses a fixed anchor set: anchors is a frozen buffer, and only `log_sigma` and `w_proj` are learned. The learnable V_θ parameters are therefore the weights and biases of those projection layers, rather than the direct positions of the wells themselves.

What else in the model is learnable ?

Everything which carries gradients is learnable. So the question becomes which quantities carry gradients.

Token embedding `E` – `nn.Embedding(vocab, d)`, the largest single block ($\sim 19M$ at vocab $V = 50257, d = 384$)

model_parf.py, lines 1064-1078

```
def compute_logits(self, h_L: torch.Tensor) -> torch.Tensor:
    """Project final hidden state to vocab logits via the read-out head.

    Uses the dedicated W_out when untied, else the tied E^T, and adds the
```

```
    optional learned output bias.  Diagnostics that reconstruct logits by
    hand should call this (rather than `h_L @ E.weight.T`) so the bias /
    untied head are respected.
    """
    if self.lm_head is not None:
       logits = self.lm_head(h_L)
    else:
       logits = h_L @ self.E.weight.T
    if self.out_bias is not None:
       logits = logits + self.out_bias
    return logits
```

The token embedding E learns both the input placement `h_0` and the decoding of `h_L`.

What else carries gradients (everything in this list is trainable)

1. Token embedding E: nn.Embedding(vocab, d). The largest single block (~19 million parameters for vocabulary 50,257 and $d = 384$). It is tied to the output head via $\text{logits} = h_L @ E.\text{weight}.T$. E learns both the input placement of h_0 and the decoding of h_L.

2. Positional embedding P: nn.Parameter(max_len, d).

3. The ξ module (`MultiChannelXi`): the learnable raw_alpha parameter, which maps to the EMA decay coefficients seen in logs (e.g. $\alpha = [0.145, \ldots, 0.989]$). These coefficients learn the context timescales that condition the wells.

4. V_θ (Gaussian wells): mu_proj, a_proj, and w_proj — the energy landscape.

5. V_ϕ (pairwise PARF interaction): the two-body force. MLP / structural kernels and routing projections such as W_ℓ, W_θ, and theta_w_q/s/d. A substantial block; architecturally the SARF/PARF-style force, distinct from V_θ.

6. Fock register machinery (v2.1): register_embed ($M \times d$ learned register initialisation); creation and destruction gates; per-register query, key, and value projections (W_Q, W_K, W_V); per-register $\log \tau$; the reverse channel ($W_{Q,\text{rev}}, W_{K,\text{rev}}, W_{V,\text{rev}}$) and reverse_channel_scale.

7. Dynamics scalars: raw_m_bias and raw_gamma (learnable when learn_mgamma is enabled); `raw_logfreq_alpha` (surprisal-mass scale); raw_v_phi_scale. (In the phase-5 log, $\gamma = 0.300$ is constant, so γ appears fixed for that specific run.)

8. `LayerNorm` affine parameters — specifically those for `ln_after_step`.

How exactly the token embedding learns those?

When the LM head is *tied*, a single matrix $E \in \mathbb{R}^{V \times d}$ participates in the computation graph at two distinct points and receives gradients from both.

Forward pass – two uses of the same parameter:

- Input path: `h_0[t] = E[x_t] + P[t]` (row lookup, no matmul)
- Output path: `logits = h_L @ E.T` (full matmul against all V rows)

Backward pass – two gradient contributions that get summed:

PyTorch's autograd accumulates gradients. Because `E.weight` appears in two places in the computation graph, its `.grad` after `loss.backward()` is the sum of two terms :

1) from the *output path* (decoding h_L)

$$(\frac{\partial L}{\partial E_v})_{\text{output}} = \sum_t \frac{\partial L}{\partial \text{logit}_{t,v}} h_t^* \text{ (out.1)}$$

That pushes each row E_v of the embedding matrix E to be a good linear classifier of the final hidden state – in other words row v should have high dot product h_L (aka h^*) when v is the correct next token. This gradient flows through the entire Verlet dynamics (all L layers of V_θ and V_ϕ before reaching E), so it is the "deep" gradient.

Clarification: E refers to the token embedding ($E \in R^{V \times d}$); the positional embedding P is a separate component.

When the LM head is tied, the matrix E is used in two places:

• Input path: $h_0[t] = E[x_t] + P[t]$. This is a simple row lookup, not a matrix multiplication.

• Output path: $\text{logits} = h_L @ E^T$. This is a full matrix multiplication of the final hidden state against all V rows of the embedding matrix.

Because $E.\text{weight}$ appears in two places, its total gradient after loss.backward() is the sum of two distinct contributions.

Contribution 1 — from the output path (decoding h_L):

$$\frac{\partial L}{\partial E_v}|_{\text{output}} = \sum_t \frac{\partial L}{\partial \text{logit}_{t,v}} \cdot h_{L,t}$$

This gradient pushes each row E_v to act as a good linear classifier for the final hidden state. Specifically, row v should have a high dot product with h_L when token v is the correct next token. Because this gradient flows back through the entire Verlet dynamics (all L layers of V_θ and V_ϕ), it is a "deep" gradient.

Contribution 2 — from the input path (placement of h_0):

$$\frac{\partial L}{\partial E_v}|_{\text{input}} = \sum_{t:x_t=v} \frac{\partial L}{\partial h_{0,t}}$$

This gradient pushes row E_v to position the initial hidden state h_0 at a specific location in R^d from which the downstream dynamics (Verlet integration through V_θ, V_ϕ) can most effectively evolve it toward the correct prediction. Like the output path, this gradient is also "deep" as it flows through all L layers via backpropagation through the integrator.

The tension

Each row E_v must simultaneously satisfy:

• As input direction: when token v appears at position t, place $h_{0,t}$ so the dynamics can steer it toward the correct next token.

• As output direction: the dot product $h_L \cdot E_v$ should be high when v is the target and low otherwise.

These are different objectives sharing the same parameter. For frequent tokens, both gradients are strong and consistent — the row gets plenty of signal from both paths and can find a compromise. For rare tokens, the input gradient fires

rarely (the token barely appears in training batches), so the output gradient dominates — but the output gradient itself is weak because the rare token is almost never the correct target. The row E_v for a rare token ends up under-trained from both directions.

This is exactly the root cause the D0.4 diagnostic exposed: rare tokens have CE above the uniform baseline ($\ln V$), meaning the model actively assigns them less probability than a uniform prior would. The output bias (Fix 1) and untied head (Fix 2) break this tension:

Setup	Input role	Output role	Rare-token problem
Tied (baseline)	E_v	E_v	Both roles fight over the same under-trained row
Tied + bias (Fix 1)	E_v	$E_v + b_v$	Bias carries the unigram prior; E_v only needs to encode contextual deviation
Untied (Fix 2)	E_v	W_out,v	Completely independent parameters; each is trained by only one gradient source

In the current multi-head run (`TIE_EMBEDDINGS=True`, `USE_OUTPUT_BIAS=True`), the bias $b_v = \log p_{\text{unigram}}(v)$ absorbs the frequency prior at step 0, so E no longer needs to encode "how common is token v" in its geometry — it can focus on encoding semantic placement for the input and contextual discrimination for the output.

Note: distinction between "deep" and "shallow" gradient

Let us trace the full path of the gradient in (out.1) to make this statement concrete.

The computation graph from E to the loss (output path):

```
E[x_t] → h_0[t] = E[x_t] + P[t]
                ↓
        Verlet layer 1 (V_theta, V_phi, xi, m, gamma, ...)
                ↓
        Verlet layer 2
                ↓
               ...
                ↓
         Verlet layer L
                ↓
             h_L[t]
                ↓
 logit_{t,v} = e_v^T h_L[t]    ← E appears again here
                ↓
  softmax → cross-entropy → L
```

When loss.backward() runs, PyTorch computes $\frac{\partial L}{\partial E_v}$ by summing contributions from every node in this graph where E was used as eq. (out.1) indicates. This term is "deep" because `h_{L,t}` (aka h_t^*) itself depends on E (through the input path `h_0 = E[x_t] + P[t]`). So the full chain rule for this contribution passes through:

E (embed) → h_0 → layer 1 → h_1 → ⋯ → layer L → h_L → logits Eh_L → CE → L

So that's L Verlet integration steps in the backward path. Each Verlet step involves `V_theta, V_phi, xi, m, gamma` - the gradient flows backward through all of them. For comparison let us consider the "shallow" gradient (the input-path comparison, discussed in the previous paragraph) :

$\left.\frac{\partial L}{\partial E_v}\right|_{\text{input}} = \sum_{t:\, x_t = v} \frac{\partial L}{\partial h_{0,t}}$ (in.1)

Eq. (in.1) is the **input-path** contribution: the gradient arrives at E by row-selection at $h_0[t] = E[x_t] + P[t]$, after $\partial L / \partial h_{0,t}$ has already back-propagated through all L layers. It is not the output matmul path of (out.1). The factor $\partial L / \partial h_{0,t}$ still travelled through all L layers to get there.
The "deep" gradient is the one that trains E's rows to be good output classifiers – it pushes E[v] to align with h_L whenever token v is the correct next token. This gradient carries information about *what the dynamics did* across all L layers (because it flows through them), which means:

1) It is the gradient that couples E to V_theta, V_phi, and all the dynamical parameters. Changes to the potentials affect h_L, which changes how E needs to orient its rows.

2) It is more susceptible to gradient pathologies (vanishing, exploding) because it passes through L sequential nonlinear transformations. This is exactly why the Plummer softening, gradient clipping, and the stability hierarchy matter – a 1/r spike in V_phi at any layer can amplify this gradient.

3) With a *tied* head, this same E that receives this deep gradient is also the embedding used to *create* h_0. So the deep gradient (which wants E[v] to be a good decoder of h_L) and the input-path gradient (which wants E[v] to be a good initial placement for token v) are pulling on the *same parameters* – creating the tension that the untied-head and output-bias fixes address.

Note: gradient of the semantic mass
Gradients are computed for the semantic mass parameters as described below:
The mass is learnable (`learn_mgamma=True` is the default in `PARFConfig`). There are two trainable parameters involved:

1) `raw_m_bias` – a scalar `nn.Parameter` that sets the baseline mass. It is stored in a "raw" (unconstrained) space; the actual mass is recovered via `softplus(raw_m_bias) + 1e-3` to guarantee positivity.

2) `raw_logfreq_alpha` – a scalar that controls how strongly token frequency modules mass. With `mass_mode = 'logfreq'` the per-token mass is computed as:

```
m(x, t) = softplus(raw_m_bias + alpha * surprisal[x_t]) + 1e-3
```

where alpha = softplus(raw_logfreq_alpha) and surprisal[x_t] is precomputed log-frequency surprisal for token type x_t. Rare tokens get higher surprisal, and if alpha is positive, they get heavier mass – making them change direction more slowly in the Verlet dynamics (physical intuition: rare tokens are harder to push around).

Both raw_m_bias and raw_logfreq_alpha have requires_grad=True, so they participate fully in the backward pass and receive gradient updates from AdamW like every other parameter.

What the gradient "means" for mass-

The mass $\mathfrak{m}$ enters the Verlet update as the denominator of the acceleration. Schematically,

$$h^{(l+1)} = h^{(l)} + v\Delta t + \frac{1}{2}\frac{F}{m}(\Delta t)^2$$

So $\partial L / \partial m$ tells the optimizer – "if you made this token heavier (harder to move), would the loss go up or down?" If a rare token is being pushed too aggressively by V_ϕ / V_θ forces and overshooting, the gradient will push its mass upward. If a token is too sluggish and not reaching a good attractor, the gradient pushes mass downward. Here F is the force $-\nabla_h(V_\theta + V_\phi)$; the displayed Verlet step is schematic (the production integrator is Stoermer-Verlet / BAOAB, not this one-line map).

However, note that `gamma` (the damping coefficient) is controlled by the same `learn_mgamma` flag. In the current experiment, the logs show gamma=0.300 is constant – but that's because it converged to that value, not because it is frozen. Both mass and gamma are free to learn.

The conceptual point

The force the integrator actually follows is the sum of two terms. In code (model_parf_sparse.py, lines 456–457):

```
if _has_analytical_grad(self.V_theta):
    f_theta = -self.V_theta.analytical_grad(xi_now, h_in)  # (B, T, d)
```

$$f = -\nabla_h V_\theta - \nabla_h V_\phi$$

The trajectory the loss is computed on depends on: where E places h_0; what context ξ (learned EMAs) feeds the wells; the pairwise V_ϕ forces; which Fock particles get spawned or destroyed; the mass/damping parameters that set the integration; and the tied E that decodes h_L. Gradients from the NTP loss flow back through all of these components.

The semantic energy landscape is the distinctive and interpretable component that defines single-particle attractor basins. It is not the sole locus of learning. The model would "learn only through the wells" only if everything else was frozen — specifically the embedding, V_ϕ, Fock gates, ξ, and mass/γ. The fock_gaussian_sarf phase-5 configuration does not freeze these elements.

Note on Why Tied LM Head worsens the Perplexity score

The Problem: Tied head forces the dynamics to encode the frequency prior

For details refer to **D0** free-diagnostics discussed in Section 8 and proposed fix presented in Section 9 of [7].
So with tied embeddings the logit for token v at position t is:

$\ell_v = h_L \cdot e_v$
where e_v is the same row of the embedding matrix E used for *both* input lookup and output scoring.
This creates two compounding problems

This is about the **D0.4** long-tail diagnostic finding. With tied embeddings (`TIE_EMBEDDINGS=True`), the logit for token v at position t is

$$\text{logit}_v = h_L \cdot e_v$$

where e_v is the same row of the embedding matrix E used for both input lookup and output scoring, and h_L is the hidden state at the final layer.

The problem: tied head forces the dynamics to encode the frequency prior

This creates two compounding problems:

1. Encoding the frequency prior in h_L. To predict frequent tokens as more likely than rare ones (even without context), the model must rotate h_L toward a "frequent-token direction" — the direction having a high dot product with the embeddings of frequent tokens. This conflicts with the direction h_L needs to encode semantic or contextual information. The model is forced into a compromise.

2. Double duty for undertrained rare-token embeddings. Rare tokens appear infrequently, so their rows in E receive very few gradient updates. Because embeddings are tied, the same poorly trained vector e_v must serve two roles: the input representation (when token v is in the input) and the scoring direction (when the model predicts v as output). Consequence: the model cannot distinguish a rare token v from a semantically different but equally rare token u within the output space.

D0.4 evidence

Quintile	Mean CE (nats)	Reading
Q0 (rarest)	13.16	Worse than uniform (ln V = 10.83)
Q1	11.49	Worse than uniform
Q2	10.69	~uniform
Q3	8.16	Slightly better
Q4 (most frequent)	4.54	Where all real learning lives

Everything outside the top quintile is at or above uniform-prior loss — the model assigns rare targets less than $1/V$ probability. The dynamics have learned something useful, but the tied head is destroying that signal for 80% of the vocabulary.

Why untying fixes it

With TIE_EMBEDDINGS=False, the model allocates a separate W_{out} ($V \times d$) for the output:

$$\text{logit}_v = h_L \cdot W_{\text{out}}[v] + b_v$$

Now $W_{\text{out}}[v]$ only needs to point in the direction of h_L when the context predicts token v — it does not also have to serve as the input representation for v. The input embedding E and the output projection W_{out} receive independent gradients, so rare tokens can develop well-conditioned output directions even with few training occurrences.

The output bias b_v (initialised to log-unigram-frequency) is the cheap first step — it absorbs the frequency prior so the dynamics stop wasting capacity on it. Untying is the deeper fix: it decouples the rare-token output directions from their undertrained input embeddings entirely. The full diagnostic is in [7]
(`companion_notes/Xi_Bottleneck_Diagnosis_Phase5.md`), sections 8.4 and 9.1–9.3.
Here is a self-contained mathematical analysis of why the tied head is structurally suboptimal for our architecture.

1. Setup and notation

Symbol	Meaning
V	vocabulary size (50,257)
d	hidden dimension (384), $d \ll V$
E ∈ R^{V×d}	input embedding matrix
h_L ∈ R^d	final hidden state from the conservative dynamics
v*	target token at a given position
p(v | c)	true conditional distribution given context c

Tied head: logits $z = Eh_L$, so $z_v = e_v \cdot h_L$.

Untied head: logits $z = Wh_L + b$, where $W \in R^{V \times d}$ is a separate output matrix and $b \in R^V$ is a bias.

2. The column-space bottleneck

Both tied and untied logit vectors live in a d-dimensional subspace of R^V. Specifically, the set of realisable logit vectors is:

$$Z_{\text{tied}} = \text{col}(E) = \{E_h : h \in R^d\}$$

$$Z_{\text{untied}} = \text{col}(W) = \{W_h : h \in R^d\}$$

Both are d-dimensional (assuming full rank). The difference is which d-dimensional subspace they span. With tying, the subspace is dictated by the input embedding objective. With untying, it is free to be optimised purely for the output objective.

3. The optimal logit vector and the projection error

For context c, let the Bayes-optimal logit vector be:

$$z^*(c) = \log p(\cdot \mid c) + \text{const} \in R^V$$

(The constant cancels in the softmax.) The model's cross-entropy excess over Bayes-optimal is, to second order around the optimum:

$$CE_{\text{excess}}(c) \approx \frac{1}{2}(z^* - \hat{z})^\top \text{diag}(p^*)(z^* - \hat{z})$$

where $\hat{z}$ is the model's logit vector and $p^* = \text{softmax}(z^*)$. This is the Fisher-weighted squared error — the cost of logit mis-specification is proportional to the weighted projection error of z^* onto the model's realisable subspace.

Decompose z^* using the SVD of the embedding matrix. Let $E = U\Sigma Q^\top$ (thin SVD: $U \in R^{V \times d}$, orthonormal columns). Then the projection of z^* onto $\text{col}(E)$ is $UU^\top z^*$, and the residual is:

$$r_E(c) = z^*(c) - UU^\top z^*(c) = (I - UU^\top)z^*(c)$$

This residual is irreducible for the tied head — no choice of h_L can reduce it. For the untied head with its own thin SVD $W = \tilde{U}\tilde{\Sigma}\tilde{Q}^\top$ (W's factors are not those of E):

$$r_W(c) = (I - \tilde{U}\tilde{U}^\top)z^*(c)$$

Since W is trained purely to minimise output CE, it will orient $\text{col}(W)$ to minimise $E_c[\|r_W(c)\|^2_{p^*}]$, the average Fisher-weighted residual. But $\text{col}(E)$ is a compromise — it must also serve the input representation objective, so in general:

$$E_c[\|r_E(c)\|^2_{p^*}] \geq E_c[\|r_W(c)\|^2_{p^*}]$$

The tied head pays a subspace alignment penalty: it cannot rotate its d output directions to best capture the variation in $z^*(c)$ across contexts.

4. Decomposition: frequency prior + contextual signal

The optimal logit vector decomposes as:

$$z^*(c) = \log p_{\text{unigram}}(v) + \log \frac{p(v \mid c)}{p_{\text{unigram}}(v)} = f + \delta(c)$$

The first term $f = \log p_{\text{unigram}}(v) \in R^V$ is a fixed vector (context-independent). The second term $\delta(c) \in R^V$ is the contextual correction — where the semantic content lives.

Without an output bias, the model represents both components via the product Eh_L:

$$Eh_L(c) \approx f + \delta(c)$$

h_L must allocate some of its d dimensions to encode f via the projection $E^\dagger f$ (where $E^\dagger$ is the pseudoinverse). Because f has significant energy outside the column space of E, this encoding is lossy.

With output bias $b = f$: the bias absorbs f exactly. The dynamics only need $Eh_L(c) \approx \delta(c)$. All d dimensions can then serve semantic content.

Let $f_\parallel = UU^\top f$ be the component of f in the column space of E, and $f_\perp = f - f_\parallel$ the residual. The model loses $\|f_\perp\|_{p^*}^2$ as irreducible error and uses $\text{rank}(\text{proj}_{f_\parallel})$ effective dimensions of h_L to encode the frequency prior, reducing capacity for contextual information.

On OpenWebText with a Zipfian distribution, f has a high dynamic range (~14 nats). Its projection onto the top singular vectors of E (which capture semantic directions, not frequency) is poor, making $\|f_\perp\|$ large and the capacity cost significant.

5. Gradient conflict for rare tokens

For a token v with corpus count n_v, the embedding row e_v receives two types of gradient.

Input gradient (when v appears at input position s):

$$\frac{\partial L}{\partial e_v}\Big|_{\text{input}} = \frac{\partial L}{\partial h_0^{(s)}} \text{ (backprop through the dynamics)}$$

This pulls e_v toward a good starting point for the conservative dynamics — it wants tokens with similar contexts to have similar embeddings.

Output gradient (over all positions t, not only when v is the target):

$$\left(\frac{\partial L}{\partial e_v}\right)_{\text{output}} = \sum_t (p_t(v) - 1_{y_t=v}) h_L^{(t)}$$

The target terms pull e_v toward alignment with $h_L^{(t)}$ when v should be predicted; the non-target terms $p_t(v) h_L^{(t)}$ push it away when the model still assigns mass to v. Restricting the sum to $\{t: y_t = v\}$ would drop the latter.

The total gradient for e_v over a training epoch is:

$$\nabla_{e_v} L = \sum_{s: x_s = v} \left(\frac{\partial L}{\partial e_v}\right)_{\text{input}} + \sum_t \left(\frac{\partial L}{\partial e_v}\right)_{\text{output},t}$$

Frequent tokens (n_v large): both sums have many terms, the compromise direction is statistically well-determined, and the cosine similarity between the input-optimal and output-optimal directions averages out to something workable.

Rare tokens (n_v small): both sums have few terms. The input gradient and output gradient point in essentially unrelated directions (one is determined by the dynamics Jacobian, the other by the final hidden state). Their sum is a noisy, poorly determined compromise. The resulting e_v is suboptimal for both roles.

Quantitatively: the effective gradient signal-to-noise ratio (SNR) scales as $\sqrt{n_v}$. For the rarest quintile Q0 on OpenWebText, the median token count is ~229 over 200M tokens, while the most frequent quintile Q4 has median count ~180,000:

$$\frac{\mathrm{SNR}_{Q4}}{\mathrm{SNR}_{Q0}} \sim \frac{\sqrt{180000}}{\sqrt{229}} \approx 28\times$$

6. A lower bound on the excess CE from tying

The `Q0` embeddings are ~28× noisier than `Q4`, and this noise propagates to both the input representation quality and the output scoring quality simultaneously.

With untied W_{out}, the output row w_v receives only the output gradient — no interference from the input role. Even with the same small n_v, the gradient direction is unambiguous (always some scalar times $h_L^{(t)}$), so the embedding converges to a clean output direction despite few updates.

Combining the above, the expected excess cross-entropy from tying can be bounded as:

$$\Delta CE_{\mathrm{tie}} = CE_{\mathrm{tied}} - CE_{\mathrm{untied}} \geq \Delta_{\mathrm{subspace}} + \Delta_{\mathrm{freq}} + \Delta_{\mathrm{noise}}$$

where:

$$\Delta_{\mathrm{subspace}} = \frac{1}{2}E_c\left[\|r_E(c)\|_{p^*}^2 - \|r_W(c)\|_{p^*}^2\right] \geq 0$$

$$\Delta_{\mathrm{freq}} = \frac{1}{2}\|f_\perp\|_{p^*}^2 \cdot 1[\text{no bias}]$$

$$\Delta_{\mathrm{noise}} \propto \sum_{v \in \mathrm{rare}} p(v) \cdot \frac{\mathrm{Var}[\nabla_{\mathrm{input}}]}{n_v}$$

The first two terms are deterministic geometry; the third is a statistical penalty that grows as $1/n_v$ for rare tokens.

7. Why this is worse for Fock-PARFLM than for transformers

Standard transformers have high per-layer capacity through multi-head attention and feed-forward networks, allowing the hidden state h_L to adapt to multiple roles. In contrast, Fock-PARFLM is constrained by conservative scalar potentials where a single scalar function is shared across all layers,

$$F = -\nabla_h V_\theta - \nabla_h V_\phi$$

leaving less “slack” in the hidden state and a larger frequency-prior encoding burden (Δ_{freq}). Furthermore, the use of gathered V_ϕ with a small top_k (= 8) limits context visibility compared to transformers.

8. Connection to D0.4

The diagnostic measured CE by frequency quintile:

Quintile	Mean CE	vs ln V = 10.83	Interpretation
Q0 (rarest)	13.16	+2.33 above uniform	Δ_noise dominates
Q1	11.49	+0.66 above uniform	Δ_noise + Δ_freq
Q2	10.69	~uniform	break-even
Q3	8.16	below uniform	gradient signal adequate
Q4	4.54	well below	strong signal, but Δ_subspace still present

The Q0 result, where $CE > \ln V$, is the signature of noise error (Δ_{noise}). Rare-token embeddings become noisy due to gradient interference, causing the model to assign less than uniform probability to them. This is mathematically impossible with a dedicated output direction, as any weight vector w_v with a weak positive correlation to the hidden state h_L (when v is the target) should perform better than uniform. The tied head's gradient conflict is the sole mechanism for this worse-than-uniform calibration.